Bachelor of Engineering (B.E.) in
Computer Engineering (BCT)
Electronics, Communication & Information Engineering (BEI)
Third Year Second Part
Elective II Course

Analysis of Algorithms

Chapter 1. Introduction to Algorithm Analysis

Session 1

by:

Er. Bikal Adhikari

(*B.E., M.Sc. Engg., M.Sc. ISE*)

Assistant Professor, Dept. of Electronics & Computer Engg.

Pulchowk Campus, IOE, TU

CHAPTER OUTLINE

Chapter 1. Introduction to Algorithm Analysis

5 hours

1.1 Algorithm and its properties, RAM model, time and space complexity, detailed analysis of algorithms, Concept of Aggregate Analysis

1.2 Asymptotic notations (Big-O, Big-Ω and Big-Θ), their geometrical interpretations and examples

1.3 Concept of best case, average case and worst case performance of an algorithm

1.4 Modeling algorithms by recurrence relation

1.5 Solving recurrence relation for evaluating computational complexity

- 1.5.1 Recursion tree method
- 1.5.2 Substitution method
- 1.5.3 Using masters theorem

ALGORITHM AND ITS PROPERTIES

An algorithm is a finite set of instructions that, if followed, accomplishes a particular task. It must satisfy five criteria:

a) **Input:** Zero or more externally supplied quantities.
b) **Output:** At least one result is produced.
c) **Definiteness:** Instructions must be clear and unambiguous.
d) **Finiteness:** The algorithm must terminate after a finite number of steps.
e) **Effectiveness:** Each step must be basic enough to be carried out by a person using pencil and paper.

FORMAL REPRESENTATION OF AN ALGORITHM

Step 1: Start
Step 2: Declare variables *n*, *i*, *remainder*, *limit*
Step 3: Read *n*
Step 4: Is *n* equal to 1?
    Step 4.1: If yes, continue
    Step 4.2: Otherwise, go to Step 7
Step 5: Display "NEITHER PRIME NOR COMPOSITE"
Step 6: Go to step 20
Step 7: Is *n* equal to 2?
    Step 7.1: If yes, continue
    Step 7.2: Otherwise, go to step 10
Step 8: Display "PRIME"
Step 9: Go to step 20
Step 10: Assign 2 to variable *i*
Step 11: Compute square root of *n* and assign the value to variable *limit*
Step 12: Is *i* less than or equal to *limit*?
        Step 12.1: If yes, continue
        Step 12.2: Otherwise, go to step 19
Step 13: Compute *n* mod *i* and assign the result to *remainder*

Continued…

CONTINUED…

Step 14: Is *remainder* equal to 0?
    Step 14.1: If yes, continue
    Step 14.2: Otherwise, go to step 17
Step 15: Display “COMPOSITE”
Step 16. Go to step 18
Step 17: Increment *i* by 1
Step 18: Go to Step 11
Step 19: Display "PRIME"
Step 20: Stop

Pseudo Code Representation of an Algorithm

```
Algorithm Check_Prime(n) {
    if n = 1
              return "NEITHER PRIME NOR COMPOSITE"
    if n = 2
              return "PRIME"
    for i ← 2 to ⌊√n⌋
              do
                    if n mod i = 0
                    return "COMPOSITE"
    return "PRIME"
}
```

```
CHECK-PRIME(n)
1  if n = 1
2     return "NEITHER PRIME NOR COMPOSITE"
3  if n = 2
4     return "PRIME"
5  for i ← 2 to ⌊√n⌋
6     if n mod i = 0
7        return "COMPOSITE"
8  return "PRIME"
```

RAM Model

→ The RAM(Random Access Machine) model is used for analyzing algorithms without running them on a physical machine.

→ The RAM model has the following properties:

- A simple operation(+, /, *, -, =, if) takes one time step.
- Loops and subroutines are comprised of simple operations.
- Memory is unlimited and access takes one time step(the RAM model does not consider whether data is on cache or disk.

→ Using the RAM model, you measure the running time of an algorithm by counting the number of steps an algorithm takes on a given input.

→ Despite the simplifying assumptions made by the RAM model, it works well in practice.

Analysis of Algorithm

→ An algorithm is analyzed based on following parameters:

- Time Complexity
- Space Complexity
- Network
- Power
- CPU Registers

TIME AND SPACE COMPLEXITY

Time Complexity:

→ Time complexity is defined as the amount of time taken by an algorithm to run, as a function of the length of the input.

→ It measures the time taken to execute each statement of code in an algorithm.

→ It is not going to examine the total execution time of an algorithm.

→ Rather, it gives the information about the variation (increase or decrease) in execution time when the number of operations (increase or decrease) in an algorithm.

→ Yes, as the definition says, the amount of time taken is a function of the length of input only.

Space Complexity:

→ Space complexity is the working space or storage that is required by any algorithm.

→ It is directly dependent or proportional to the amount of input that the algorithm takes.

→ To calculate space complexity, all you have to do is calculate the space taken up by the variables in an algorithm.

→ The lesser space, the faster the algorithm executes.

→ It is also important to know that time and space complexity are not related to each other.

Best Case, Worst Case and Average Case Complexity

→ Often, an algorithm's input determines the number of steps the algorithm takes to run.

→ For a given input n,

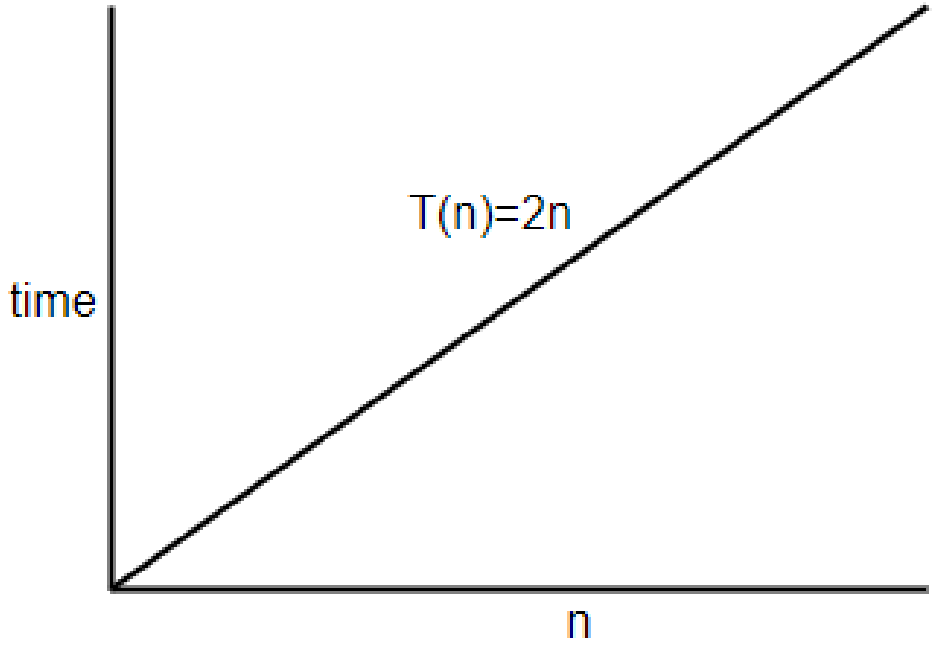
→ The best-case complexity is the minimum number of steps taken to complete.

→ The worst-case complexity is the maximum number of steps taken to complete.

→ The average-case complexity is the average number of steps taken to complete.

→ Each of these complexities defines a numerical function that represents time vs problem size. For example, for an input of size ***n***, an algorithm takes **2*n*** steps to complete in the worst case. The worst-case function for time complexity here is

$$T(n) = 2n$$



→ Worst-case complexity is normally the most useful measure.
→ This is because the best-case is often so unlikely that it isn't beneficial to think about.
→ The average-case can be useful, but it is usually difficult to determine.
→ The worst-case is both likely to happen, and easy to calculate

Time Complexity and Space Complexity of Different Algorithms

```
Algorithm Swap(p, q)
{
   temp=p;
   p=q;
   q=temp;
}
```

Time Complexity:

$f(n) = 1 + 1 + 1 = 3$

$f(n) = O(1)$

Space Complexity:

p ------- 1

q ------- 1

temp -------*1*

$S(n) = 3$

```
Algorithm Sum(A, n)
{
   S=0;
   for(i=0;i<n;i++)
   {
   S=S+A[i];
   }
   return S;
}
```

Time Complexity:

$f(n) = 1 + n+1 + n + n$

$= 3n + 2$

$\therefore f(n) = O(n)$

Space Complexity:

A ------- n

n ------- 1

S -------*1*

i -------*1*

$S(n) = n + 3$

$\therefore S(n) = O(n)$

```
Algorithm Add(A, B, C, n)
{
   for(i=0;i<n;i++)
   {
      for(j=0;j<n;j++)
      {
      C[i][j]=A[i][j]+B[i][j];
      }
   }
}
```

Time Complexity:

$f\,n) = f_1(n)*f_2(n)$

$=(n+1)*(3n+2)$

$= 3n^2 + 5n + 2$

$\therefore f\,n)=O(n^2)$

Space Complexity:

A ------- n^2

B ------- n^2

C ------- n^2

i -------1

j -------1

n -------1

$S(n) = 3n^2 + 3$

$\therefore S(n) = O(n^2)$

```
Algorithm Multiply(A,B, C, n)
{
    for(i=0;i<n;i++)
    {
        for(j=0;j<n;j++)
        {
        C[i][j]=0;
             for(k=0;k<n;k++)
             {
             C[i][j]+=A[i][k]*B[k][j];
             }
        }
    }
}
```

Time Complexity:

$f\,n) = f_1(n)*f_2(n)*f_3(n)$

$=(2n+2)*(2n+2)*(3n+3)$

$\therefore f\,n)=O(n^3)$

Space Complexity:

A ------- n^2

B ------- n^2

C ------- n^2

i -------1

j -------1

k -------1

n -------1

$S(n) = 3n^2 + 4$

$\therefore S(n) = O(n^2)$

NUMERICAL EXAMPLE

\# *Comparison of running times*

For each function $f(n)$ and time t in the following table, determine the largest size n of a problem that can be solved in time t, assuming that the algorithm to solve the problem takes $f(n)$ microseconds.

	1 second	1 minute	1 hour	1 day	1 month	1 year	1 century
$\lg n$							
$\sqrt{n}$							
n							
$n \lg n$							
n^2							
n^3							
2^n							
$n!$							

NUMERICAL EXAMPLE CONTD…

Solution:

	1 sec	1 min	1 hour	1 day	1 month	1 year	1 century
$\lg n$	2^{10^6}	$2^{6\times10^7}$	$2^{36\times10^8}$	$2^{864\times10^8}$	$2^{2.5\times10^{12}}$	$2^{3.1\times10^{13}}$	$2^{3.1\times10^{16}}$
$\sqrt{n}$	10^{12}	36×10^{14}	1296×10^{16}	7.4×10^{21}	6.7×10^{24}	9.9×10^{26}	9.9×10^{32}
n	10^6	6×10^7	36×10^8	864×10^8	2592×10^9	31536×10^9	3.1×10^{15}
$n\lg n$	62746	2.8×10^6	1.3×10^8	2.7×10^9	7.1×10^{10}	7.9×10^{11}	6.8×10^{13}
n^2	1000	7745	60000	293938	1609968	5615692	56175382
n^3	100	391	1532	4420	13736	31593	146677
2^n	19	25	31	36	41	44	51
$n!$	9	11	12	13	15	16	17

Time Complexity of Standard Control Structures

1.

```
for(i=0;i<n;i++)
{
   statement;
}
```

Time Complexity:

$f(n) = 3n + 2$

$\therefore f(n) = O(n)$

```
for(i=n;i>0;i--)
{
   statement;
}
```

Time Complexity:

$f(n) = 3n + 2$

$\therefore f(n) = O(n)$

2.

```
for(i=0;i<n;i++)
{
    for(j=0;j<n;j++)
    {
        statement;
    }
}
```

Time Complexity:

$\therefore f(n) = O(n^2)$

3.

```
for(i=0;i<n;i++)
{
    for(j=0;j<i;j++)
    {
    statement;
    }
}
```

i	j	times
0	0 ×	0
1	0	1
	1 ×	
2	0	2
	1	
	2 ×	
3	0	3
	1	
	2	
	3 ×	
..	..	..
..	..	..
n	0	*n*
	1	
	..	
	n-1	
	n ×	

$$1+2+3+\ldots+n=\frac{n(n+1)}{2}=\frac{n^2+n}{2}$$

$$\therefore\ f(n)=O(n^2)$$

Therefore, time complexity is $O(n^2)$

4.

```
P=0;
for(i=1;P<=n;i++)
{
P=P+i;
}
```

i	**P**
1	0+1=1
2	1+2=3
3	1+2+3
4	1+2+3+4
…	…
…	…
k	1+2+3+4+…+ ***k***

Assuming $P > n$

$$P = \frac{k(k+1)}{2}$$

$$\frac{k(k+1)}{2} > n$$

$$or, k^2 > n$$

$$\therefore k > \sqrt{n}$$

Therefore, time complexity is $O\left(\sqrt{n}\right)$

5.

```
for(i=1;i<n;i=i*2)
{
  statement;
}
```

i
1
$1\times2=2$
$2\times2=2^2$
$2^2\times2=2^3$
…
…
2^k

Assuming $i >= n$

$\therefore i = 2^k$

$or, 2^k >= n$

$or, 2^k = n$

$\therefore k = \log_2 n$

Therefore, time complexity is $O(\log_2 n)$

6.

```
for(i=n;i>=1;i=i/2)
{
   statement;
}
```

i
n
$n/2$
$n/2^2$
$n/2^3$
…
…
$n/2^k$

Assuming $i < 1$

$$\frac{n}{2^k} < 1$$

$$or, \frac{n}{2^k} = 1$$

$$or, n = 2^k$$

$$\therefore k = \log_2 n$$

Therefore, time complexity is $O(\log_2 n)$

7.

```
for(i=0;i*i<n;i++)
{
   statement;
}
```

$$i \times i < n$$

$$or, i \times i >= n$$

$$or, i^2 = n$$

$$\therefore i = \sqrt{n}$$

Therefore, time complexity is $O\left(\sqrt{n}\right)$

8.

```
for(i=0;i<n;i++)
{
  statement;
}
for(j=0;j<n;j++)
{
  statement;
}
```

Therefore, time complexity is $O(n)$

9.

```
P=0
for(i=1;i<n;i=i*2)
{
  P++;                          -----P=log2n
}
for(j=1;j<P;j=j*2)
{
  statement;                    -----log2P
}
```

Therefore, time complexity is $O(\log_2(\log_2 n))$

10.

```
for(i=0;i<n;i++)
{
    for(j=1;j<n;j=j*2)
    {
        statement;
    }
}
```

Therefore, time complexity is $O(n \log_2 n)$

ORDER OF GROWTH OF A FUNCTION

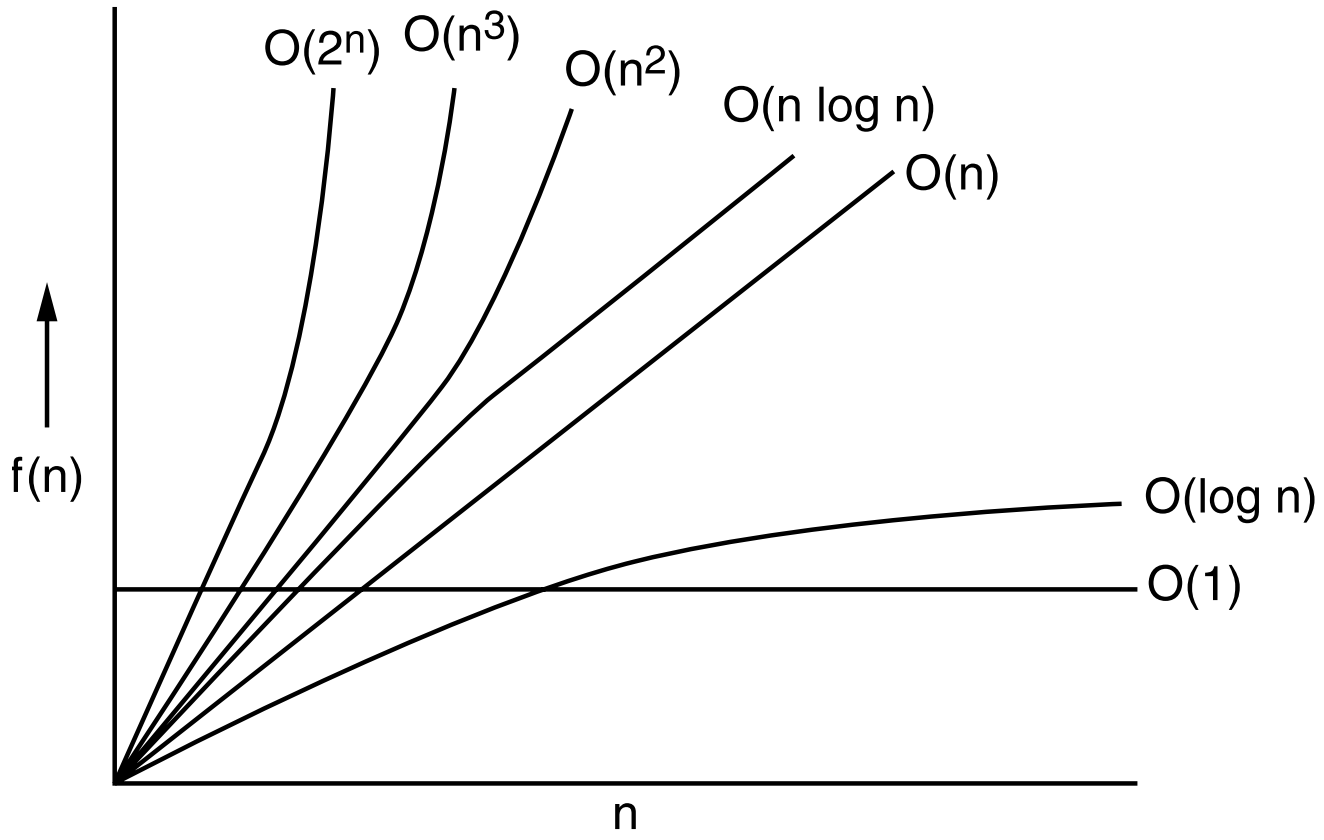
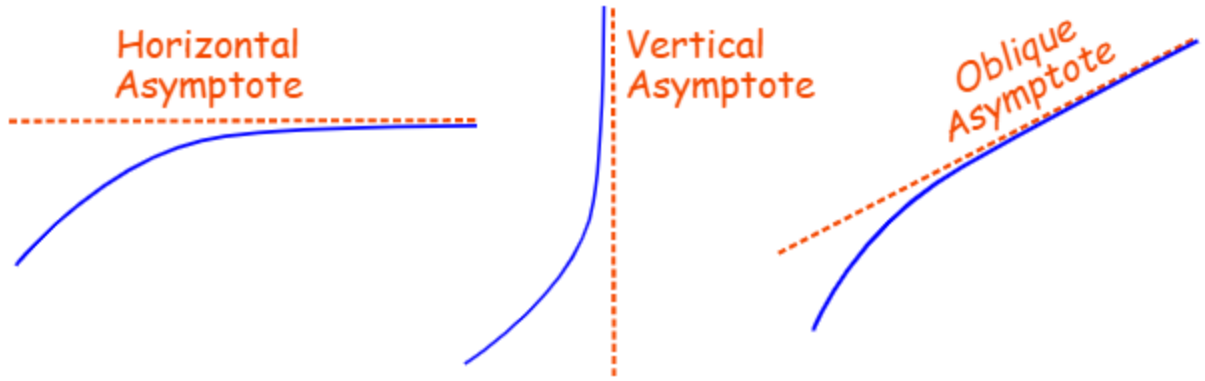


Figure The order of growth for varying problem input sizes.

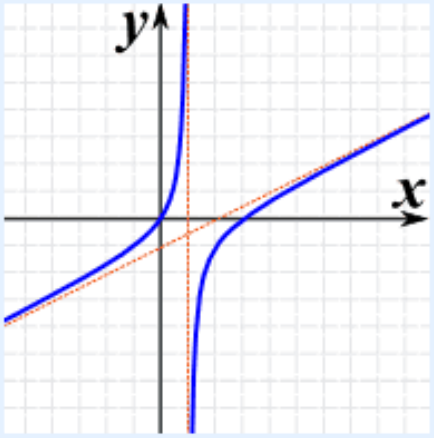
Asymptotes in Mathematics



Example: $(x^2-3x)/(2x-2)$

The graph of $(x^2-3x)/(2x-2)$ has:

- A vertical asymptote at **x=1**
- An oblique asymptote: **y=x/2 − 1**



Asymptotic Notations

$$1 < \log n < \sqrt{n} < n < n\log n < n^2 < n^3 < \ldots\ldots < 2^n < 3^n < \ldots\ldots < n^n$$

Big-Oh(O) Upper bound
Big-Omega(Ω) Lower Bound
Theta Average(Θ) Bound

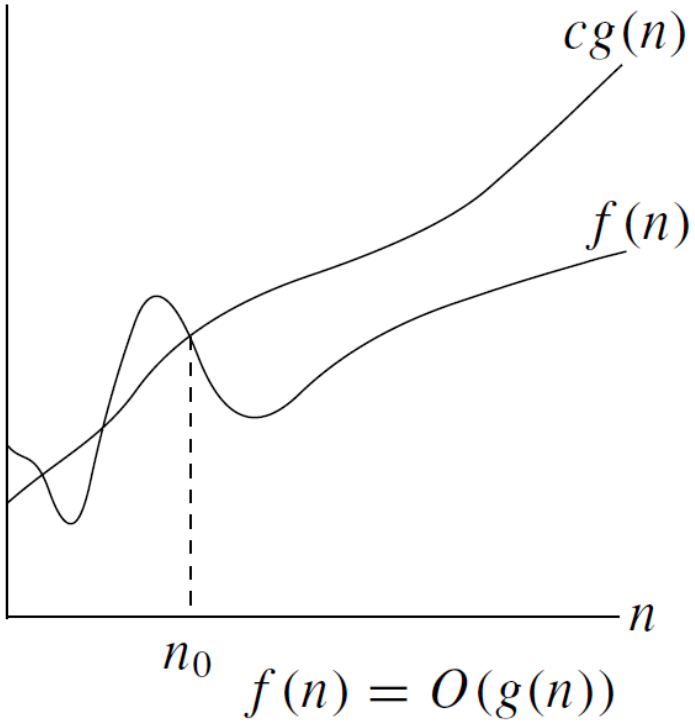
BIG-OH(O) NOTATION

The function $f(n) = O(g(n))$ *iff* $\exists$ positive constant c and n_0 such that $f(n) \leq c \times g(n) \forall n \geq n_0$

Example 1:
$f(n) = 2n + 3$

$2n + 3 \leq 2n + 3n$
or, $2n + 3 \leq 5n$
Form:
$f(n) \leq cg(n)$
True for $c = 5$ and $n > n_0$
where $n_0 = 1$

$\therefore f(n) = O(g(n)) = O(n)$



Example 2:

$f(n) = 2n + 3$

$2n + 3 \leq 2n^2 + 3n^2$

or, $2n + 3 \leq 5n^2$

Form:

$f(n) \leq cg(n)$

True for $c = 5$ and $n > n_0$

where $n_0 = 1$

$\therefore f(n) = O(g(n)) = O(n^2)$

BIG-OMEGA(Ω) NOTATION

The function $f(n) = \Omega(g(n))$ *iff* $\exists$ positive constant c and n_0 such that $f(n) \geq c \times g(n) \forall n \geq n_0$

Example 1:

$f(n) = 2n + 3$

$2n + 3 \geq 1 \times n$

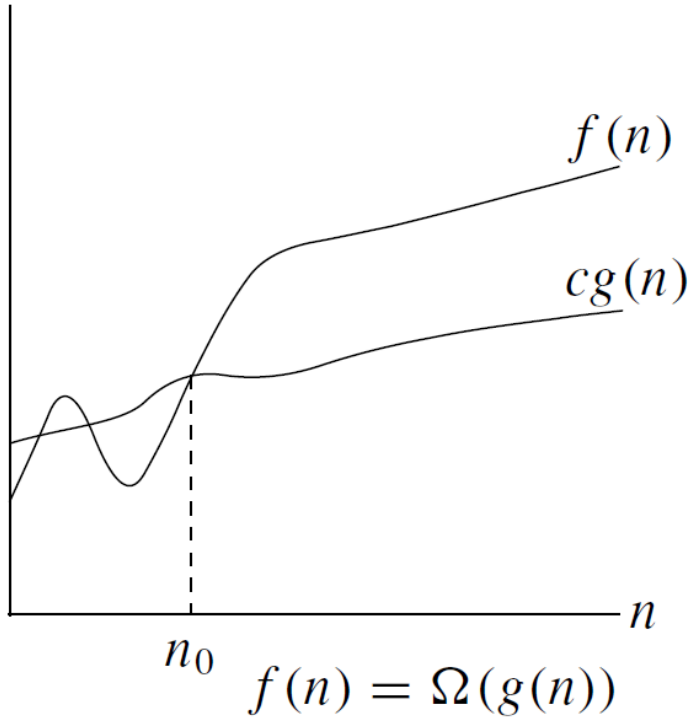
Form:

$f(n) \geq cg(n)$

True for $c = 1$ and $n > n_0$

where $n_0 = 1$

$\therefore f(n) = \Omega(g(n)) = \Omega(n)$



THETA(Θ) NOTATION

The function $f(n) = \Theta(g(n))$ iff $\exists$ positive constant c_1, c_2 and n_0 such that $c_1 \times g(n) \leq f(n) \leq c_2 \times g(n) \forall n \geq n_0$

Example 1:

$f(n) = 2n + 3$

$1 \times n \leq 2n + 3 \leq 5 \times n$

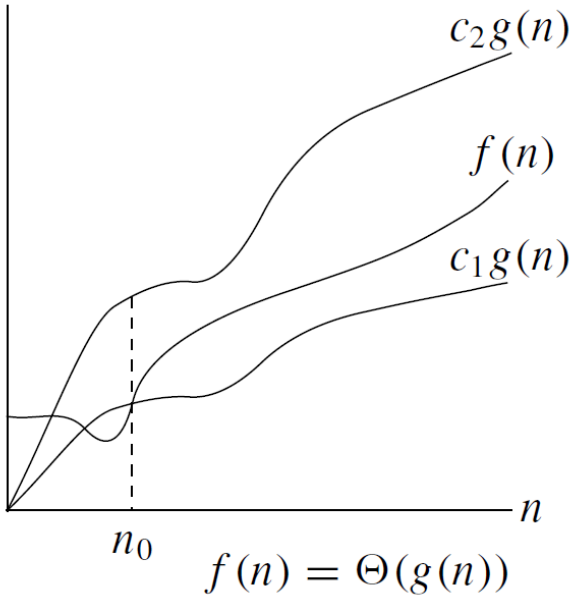
Form:

$c_1 g(n) \leq f(n) \leq c_2 g(n)$

True for $c_1 = 1, c_2 = 5$ and $n > n_0$

where $n_0 = 1$

$\therefore f(n) = \Theta(g(n)) = \Theta(n)$



EXAMPLES

Example 1:

$f(n) = 2n^2 + 3n + 4$

Solution:

For Big-Oh,

$2n^2 + 3n + 4 \leq 2n^2 + 3n^2 + 4n^2$

or, $2n^2 + 3n + 4 \leq 9n^2 \quad n \geq 1$

Form:

$\boldsymbol{f(n) \leq cg(n)}$

Where c = 9 and $n_0 = 1$

$\boldsymbol{\therefore f(n)=O(n^2)}$

For Big-Omega,

$2n^2 + 3n + 4 \geq 1 \times n^2 \quad n \geq 1$

Form:

$\boldsymbol{f(n) \geq cg(n)}$

Where c = 1 and $n_0 = 1$

$\boldsymbol{\therefore f(n)=\Omega(n^2)}$

For Theta,

$1 \times n^2 \leq 2n^2 + 3n + 4 \leq 9 \times n^2 \quad n \geq 1$

Form:

$\boldsymbol{c_1 g(n) \leq f(n) \leq c_2 g(n)}$

Where $c_1 = 1$, $c_2 = 9$ and $n_0 = 1$

$\boldsymbol{\therefore f(n) = \Theta(n^2)}$

PROPERTIES OF ASYMPTOTIC NOTATIONS

General Properties:

If $f(n)$ is $O(g(n))$ then $a*f(n)$ is $O(g(n))$

Example:

$f(n)=2n^2+5$ is $O(n^2)$

Then

$8.f(n) = 8(2n^2+5) = (16n^2+40)$ is $O(n^2)$

Reflexive Property:

If ($f(n)$ is given then $f(n) = O(f(n))$

Eg: $f(n) = n^2$ then

$f(n) = O(n^2)$

Transitive Properties:

If $f(n) = O(g(n))$ and $g(n) = O(h(n))$
then
$f(n) = O(h(n))$

Example:
$f(n) = n$, $g(n) = n^2$ $h(n) = n^3$
$n = O(n^2)$ and $n^2 = O(n^3)$
then
$n = O(n^3)$

Symmetric Properties:

If $f(n) = \Theta(g(n))$ then $g(n) = \Theta(f(n))$

Example:

$f(n) = n^2$, $g(n) = n^2$ then

$f(n) = \Theta(n^2)$

$g(n) = \Theta(n^2)$

Transpose Symmetric:
If $f(n) = O(g(n))$ then $g(n) = \Omega(f(n))$

Example: If $f(n) = n$, $g(n) = n^2$
Then
n is $O(n^2)$ and
n^2 is $\Omega(n)$

If $f(n) = O(g(n))$
and $f(n) = \Omega(g(n))$
then
$f(n) = \Theta(g(n))$

If $f(n) = O(g(n))$
and $d(n) = O(e(n))$
then
$f(n)+d(n) = O(\max(g(n),e(n)))$

Example:
$f(n) = n^2 = O(n)$
$d(n) = n = O(n^2)$
$f(n)+d(n) = n^2 + n = O(n^2)$

If $f(n) = O(g(n))$
and $d(n) = O(e(n))$
then
$f(n)*d(n) = O(g(n)*e(n))$

Example:
$f(n) = n^2 = O(n^2)$
$d(n) = n = O(n)$
$f(n)*d(n) = n^2 * n = O(n^3)$

SUMMARY OF ASYMPTOTIC NOTATIONS

When we write $f(n) = O(g(n))$,
We call it $f(n)$ is Big-oh of $g(n)$.
We mean $g(n)$ is the upper bound of the given function. In simple terms, $g(n)$ is larger than $f(n)$.
f(n) can be shown to be Big-oh of any higher function.
Example:
If $f(n) = n + 3$,
then
$f(n) = O(n)$
$f(n) = O(n^2)$
$f(n) = O(n^3)$
$f(n) = O(2^n)$
Technically, all of the above are correct, but the nearest upper bound of the given function is a linear function. i.e. $O(n)$

When we write $f(n) = \Omega(g(n))$,
We call it $f(n)$ is Big-omega of $g(n)$.
We mean $g(n)$ is the lower bound of the given function. In simple terms, $g(n)$ is smaller than $f(n)$.
$f(n)$ can be shown to be Big-omega of any lower function.
Example:
If $f(n) = n^4$,
then
$f(n) = \Omega(n^4)$
$f(n) = \Omega(n^3)$
$f(n) = \Omega(n^2)$
$f(n) = \Omega(n)$
Technically, all of the above are correct, but the nearest lower bound of the given function is a function of order 4. i.e. $O(n^4)$

When we write $f(n) = \Theta(g(n))$,
We call it $f(n)$ is theta of $g(n)$.
We mean $g(n)$ is the average bound of the given function. In simple terms, $g(n)$ is similar to $f(n)$.
$f(n)$ can be shown to be theta of any function that grows or decrease at the same rate.
Example:
If $f(n) = 10n^2$,
then
$f(n) = \Theta(n^2)$
Theta gives the tighter bound of the given function.

Concept of Aggregate Analysis

In aggregate analysis, we show that for all n, a sequence of n operations takes worst-case time $T(n)$ in total. In the worst case, the average cost, or amortized cost, per operation is therefore $T(n)/n$. Note that this amortized cost applies to each operation, even when there are several types of operations in the sequence. It is one of the three standard techniques of amortized analysis, alongside the accounting and potential methods.

Review of Stacks implemented on an Array

The INSERT operation on a stack is often called PUSH, and the DELETE operation, which does not take an element argument, is often called POP. These names are allusions to physical stacks, such as the spring-loaded stacks of plates used in cafeterias. The order in which plates are popped from the stack is the reverse of the order in which they were pushed onto the stack, since only the top plate is accessible.

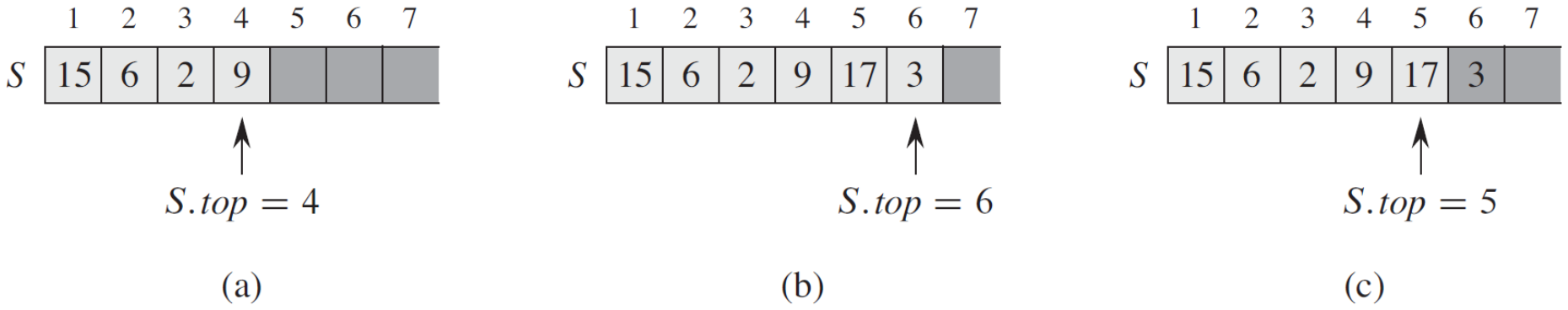


Figure: An array implementation of a stack S. Stack elements appear only in the lightly shaded positions. (a) Stack S has 4 elements. The top element is 9. (b) Stack S after the calls PUSH(S, 17) and PUSH(S, 3). (c) Stack S after the call POP(S) has returned the element 3, which is the one most recently pushed. Although element 3 still appears in the array, it is no longer in the stack; the top is element 17. **Source: (Cormen et al., 2022)**

As figure shows, we can implement a stack of at most *n* elements with an array *S*[1…..*n*]. The array has an attribute *S.top that indexes the most recently* inserted element. The stack consists of elements *S*[1…. *S.top*], *where S.*1 *is the* element at the bottom of the stack and *S.top is the element at the top.* When *S.top* = 0, *the stack contains no elements and is empty. We can test to* see whether the stack is empty by query operation STACK-EMPTY. If we attempt to pop an empty stack, we say the stack *underflows, which is normally an error.* If *S.top exceeds n, the stack overflows. (In our pseudocode implementation, we* don't worry about stack overflow.)

EXAMPLES OF AGGREGATE ANALYSIS

Stack operation:

```
STACK-EMPTY(S)
if S.top == 0
     return true
else
     return false

PUSH(S, x):
  S.top = S.top + 1
  S[S.top] =  x

POP(S):
  if STACK-EMPTY(S)
     error "underflow"
  else  S.top = S.top - 1
        return S[S.top + 1]
```

Analysis:

Each push and pop is $O(1)$.
For a sequence of n operations, total cost $T(n)=O(n)$.
Amortized cost per operation = $T(n)/n = O(1)$.
Even if we mix pushes and pops, aggregate analysis shows each operation costs constant time on average.

Binary Incrementer

INCREMENT(*A*):

```
    i = 0
    while i < k and A[i] == 1
        A[i] = 0
        i = i + 1
     if i < k
        A[i] = 1
```

$k = 8$

A(k-1) is MSB and A0 is LSB

A7	A6	A5	A4	A3	A2	A1	A0
0	0	0	0	0	0	0	0

8 bit register

0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	1
0	0	0	0	0	0	1	0
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
1	1	1	1	1	1	1	1

Analysis

Worst-case:

flipping all k bits $\rightarrow O(k)$.

Over n increments, LSB(A0) flips at most n times, A1 flips at most $n/2$ times, A2 flips at most $n/2^2$ times, and so on. Therefore,

Total flips $= n + n/2 + n/2^2 + \ldots\ldots + 1 \leq n/(1\text{-}1/2) = 2n$.

So, total cost $T(n) = O(n)$.

Amortized cost per increment $= T(n)/n = O(1)$.

Thank You!!!

Bachelor of Engineering (B.E.) in
Computer Engineering (BCT)
Electronics, Communication & Information Engineering (BEI)
Third Year Second Part
Elective II Course

Analysis of Algorithms

Chapter 1. Introduction to Algorithm Analysis
Session 2 (Review Only Session)

by:

`Er. Bikal Adhikari

(*B.E., M.Sc. Engg., M.Sc. ISE*)

Assistant Professor, Dept. of Electronics & Computer Engg.

Pulchowk Campus, IOE, TU

CHAPTER OUTLINE

Chapter 1. Introduction to Algorithm Analysis

5 hours

1.1 Algorithm and its properties, RAM model, time and space complexity, detailed analysis of algorithms, Concept of Aggregate Analysis

1.2 Asymptotic notations (Big-O, Big-Ω and Big-Θ), their geometrical interpretations and examples

1.3 Concept of best case, average case and worst case performance of an algorithm

1.4 Modeling algorithms by recurrence relation

1.5 Solving recurrence relation for evaluating computational complexity

- 1.5.1 Recursion tree method
- 1.5.2 Substitution method
- 1.5.3 Using masters theorem

Review Materials

Recurrence Relation

→ A recurrence relation for the sequence a_0, a_1,……., a_n is an equation that relates a_n to certain of its predecessors a_0, a_1,…..,a_{n-1}.

→ Initial conditions for the sequence a_0, a_1,….. are explicitly given values for a finite no. of terms of the sequence.

→ For Example:

a. Fibonacci sequence: 1 1 2 3 5 8 13 21 34 55 and so on.

⇒ The recurrence relation for Fibonacci sequence is

$$f_n = f_{n-1} + f_{n-2},\ n \geq 3$$

and initial conditions

$$f_1 = 1,\ f_2 = 1$$

f_1	f_2
1	1

Recurrence Relation

→ A recurrence relation for the sequence a_0, a_1,……., a_n is an equation that relates a_n to certain of its predecessors a_0, a_1,…..,a_{n-1}.

→ Initial conditions for the sequence a_0, a_1,….. are explicitly given values for a finite no. of terms of the sequence.

→ For Example:

a. Fibonacci sequence: 1 1 2 3 5 8 13 21 34 55 and so on.

⇒ The recurrence relation for Fibonacci sequence is

$$f_n = f_{n-1} + f_{n-2},\ n \geq 3$$

and initial conditions

$$f_1 = 1,\ f_2 = 1$$

f_1	f_2	f_3
1	1	2

Recurrence Relation

→ A recurrence relation for the sequence a_0, a_1,……., a_n is an equation that relates a_n to certain of its predecessors a_0, a_1,…..,a_{n-1}.

→ Initial conditions for the sequence a_0, a_1,….. are explicitly given values for a finite no. of terms of the sequence.

→ For Example:

a. Fibonacci sequence: 1 1 2 3 5 8 13 21 34 55 and so on.

⇒ The recurrence relation for Fibonacci sequence is

$$f_n = f_{n-1} + f_{n-2},\ n \geq 3$$

and initial conditions

$$f_1 = 1,\ f_2 = 1$$

f_1	f_2	f_3	f_4
1	1	2	3

Recurrence Relation

→ A recurrence relation for the sequence a_0, a_1,……., a_n is an equation that relates a_n to certain of its predecessors a_0, a_1,…..,a_{n-1}.

→ Initial conditions for the sequence a_0, a_1,….. are explicitly given values for a finite no. of terms of the sequence.

→ For Example:

a. Fibonacci sequence: 1 1 2 3 5 8 13 21 34 55 and so on.

⇒ The recurrence relation for Fibonacci sequence is

$$f_n = f_{n-1} + f_{n-2},\ n \geq 3$$

and initial conditions

$$f_1 = 1,\ f_2 = 1$$

f_1	f_2	f_3	f_4	f_5
1	1	2	3	5

Recurrence Relation

→ A recurrence relation for the sequence a_0, a_1,……., a_n is an equation that relates a_n to certain of its predecessors a_0, a_1,…..,a_{n-1}.

→ Initial conditions for the sequence a_0, a_1,….. are explicitly given values for a finite no. of terms of the sequence.

→ For Example:

a. Fibonacci sequence: 1 1 2 3 5 8 13 21 34 55 and so on.

⇒ The recurrence relation for Fibonacci sequence is

$$f_n = f_{n-1} + f_{n-2},\ n \geq 3$$

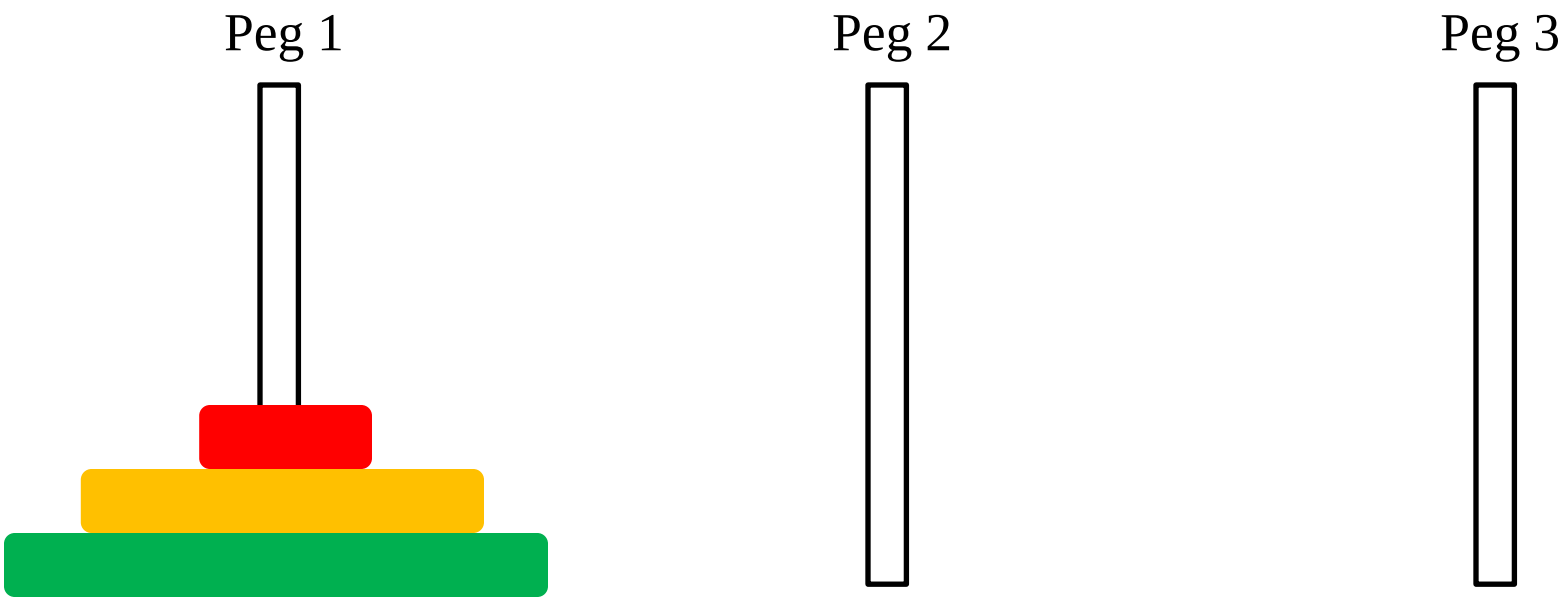
and initial conditions

$$f_1 = 1,\ f_2 = 1$$

f_1	f_2	f_3	f_4	f_5	f_6
1	1	2	3	5	8

and so on…

b. Tower of Hanoi:



Tower of Hanoi, is a mathematical puzzle which consists of three towers (pegs) and more than one disks of different size is stacked in peg 1 as shown in the above figure.

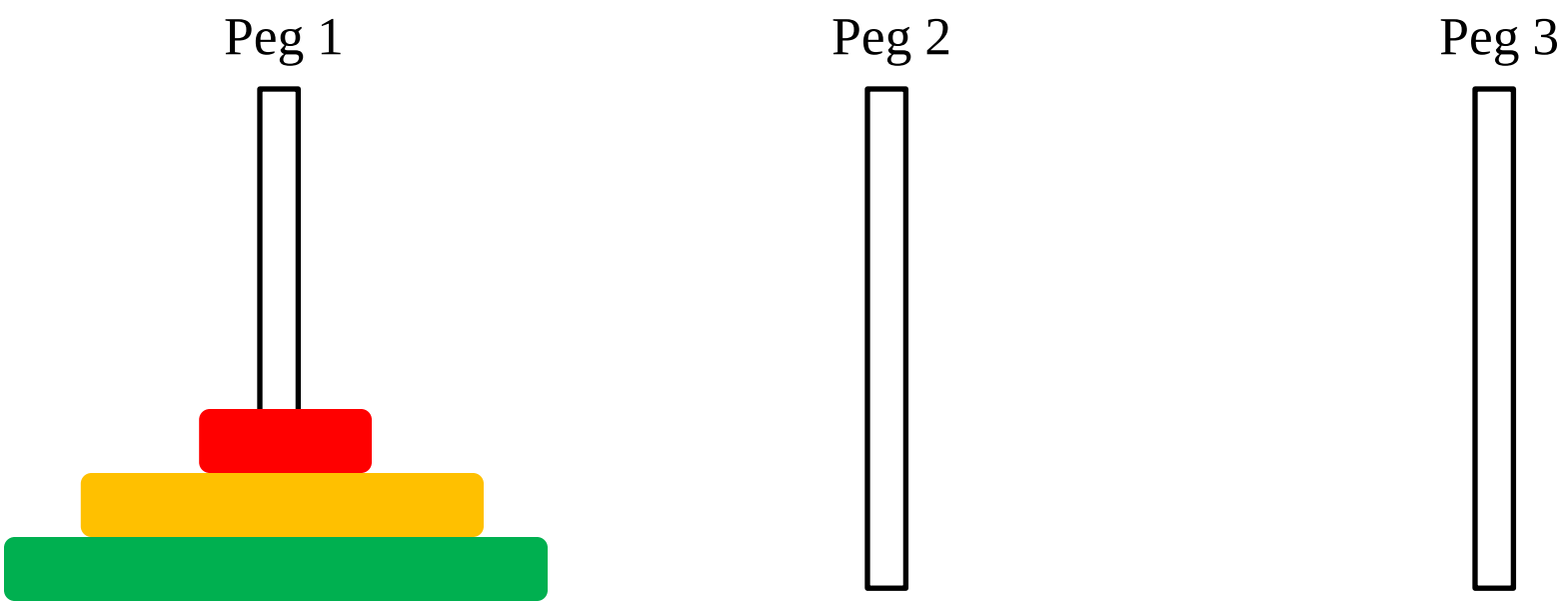
→ These disks are of different sizes and stacked upon in an ascending order, i.e. the smaller one sits over the larger one. There are other variations of the puzzle where the number of disks increase, but the tower count remains the same.

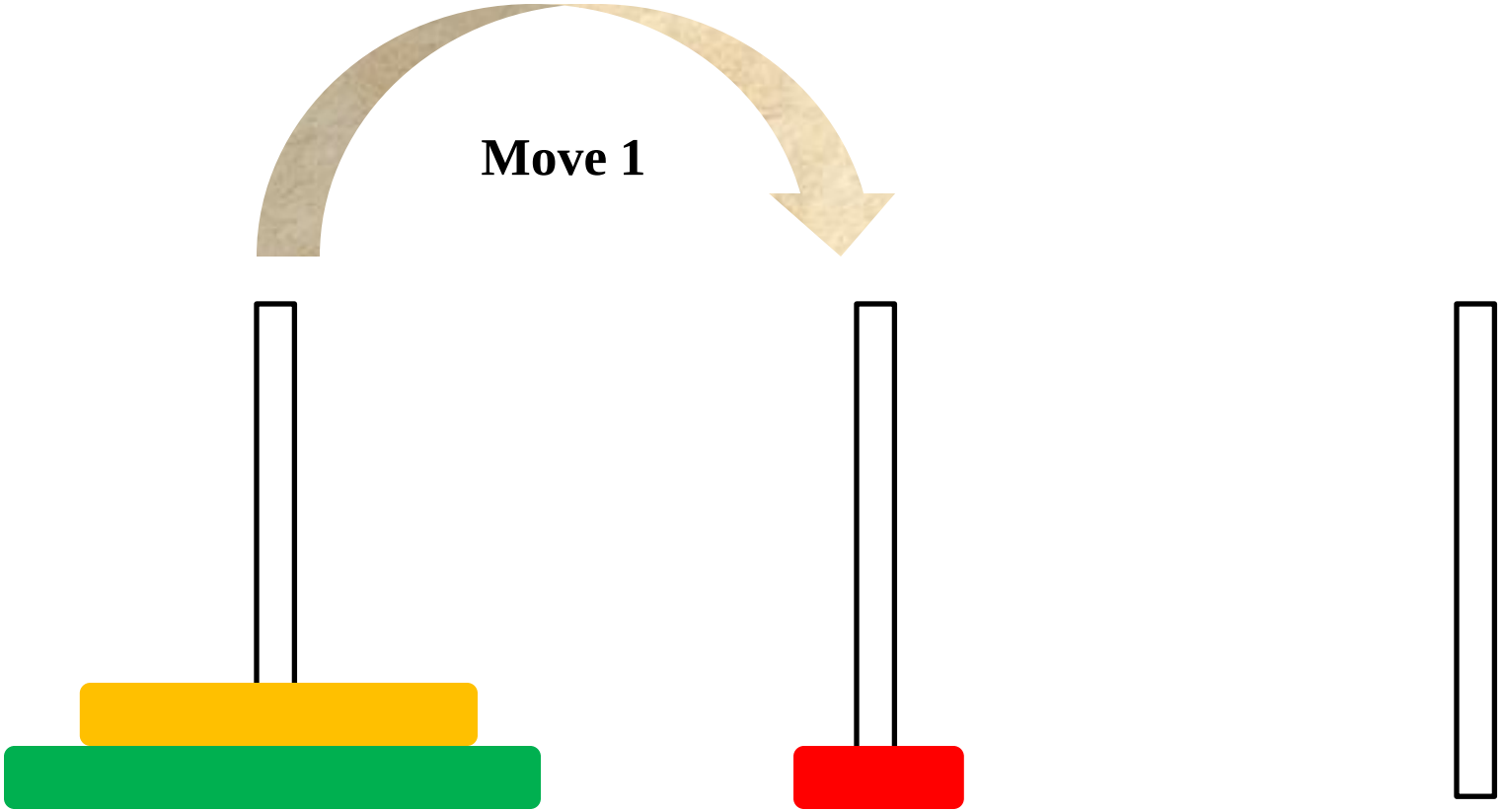
Rules:

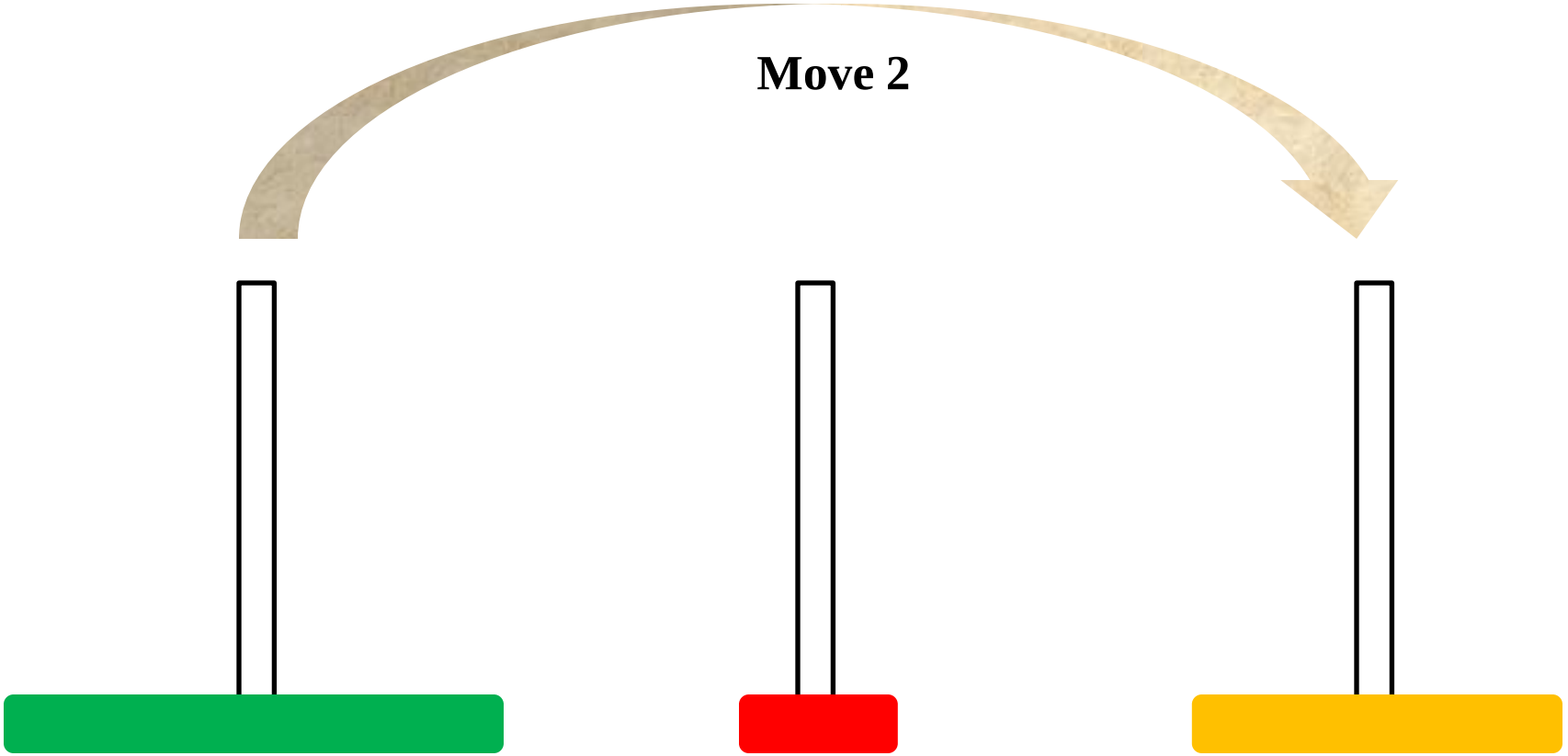
The mission is to move all the disks to some another tower**(for example: moving disks from peg1 to peg 2)** without violating the sequence of arrangement. A few rules to be followed for Tower of Hanoi puzzle are:

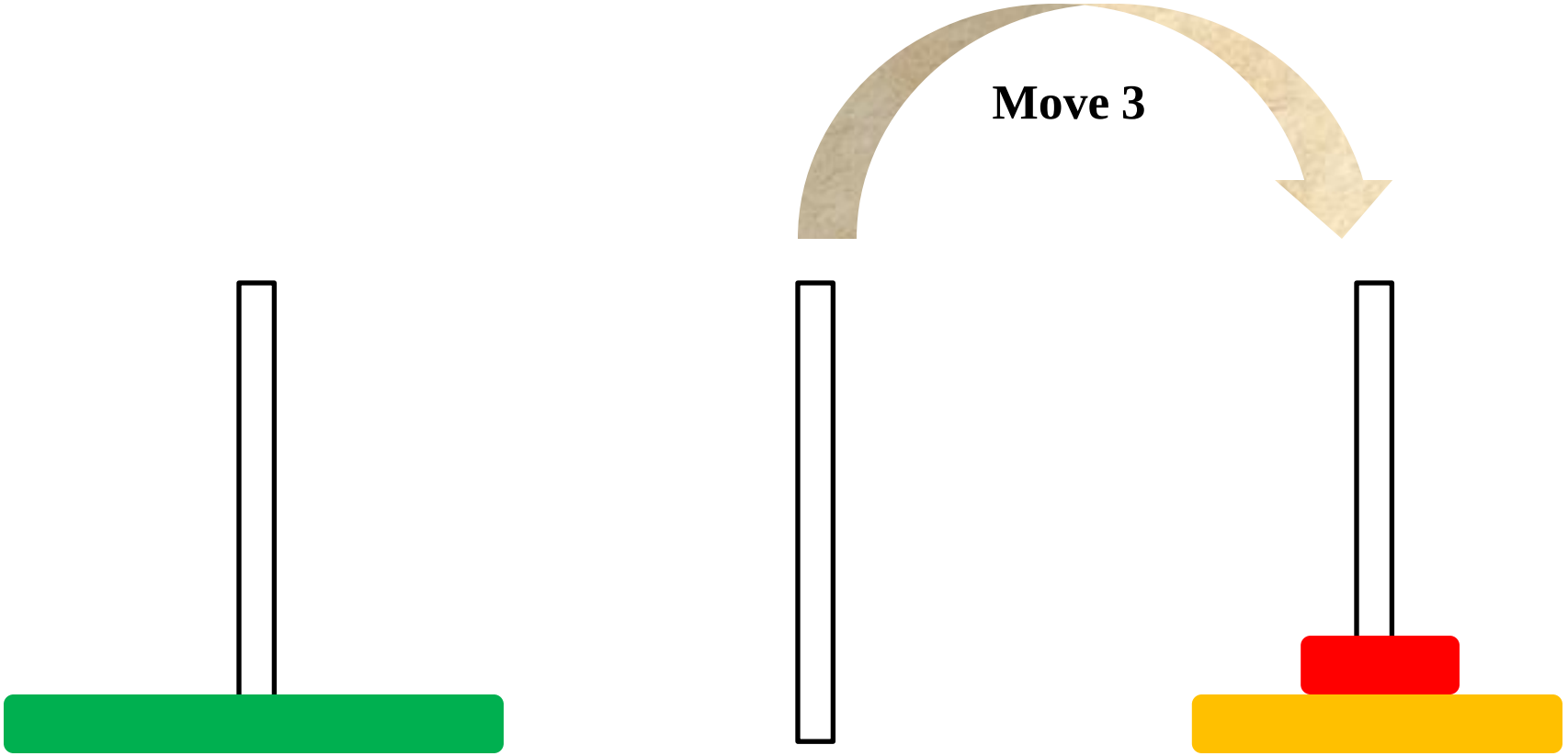
- Only one disk can be moved among the towers at any given time.
- Only the "top" disk can be removed.
- No large disk can sit over a small disk.

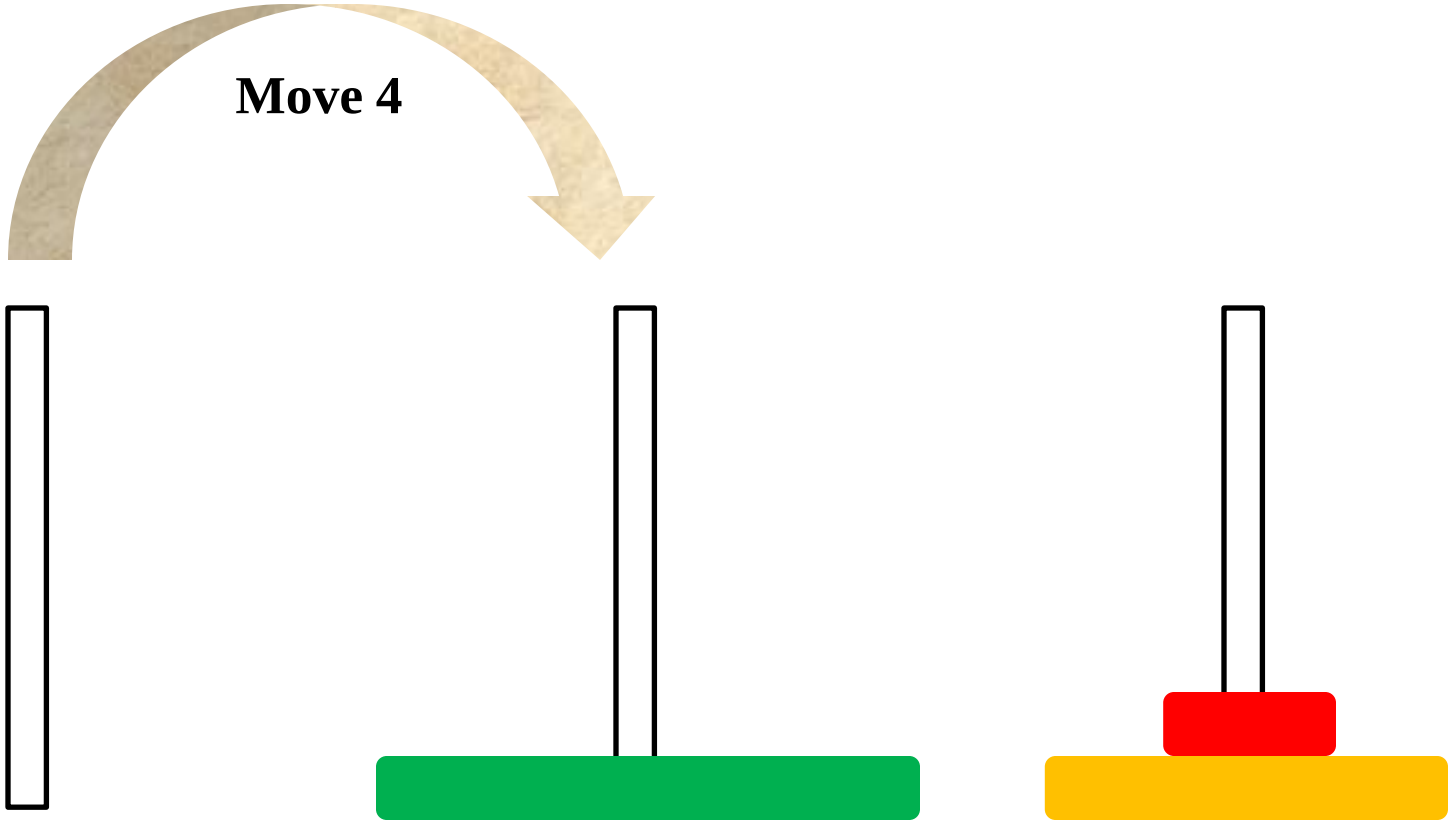
→ Following is an animated representation of solving a Tower of Hanoi puzzle with three disks.

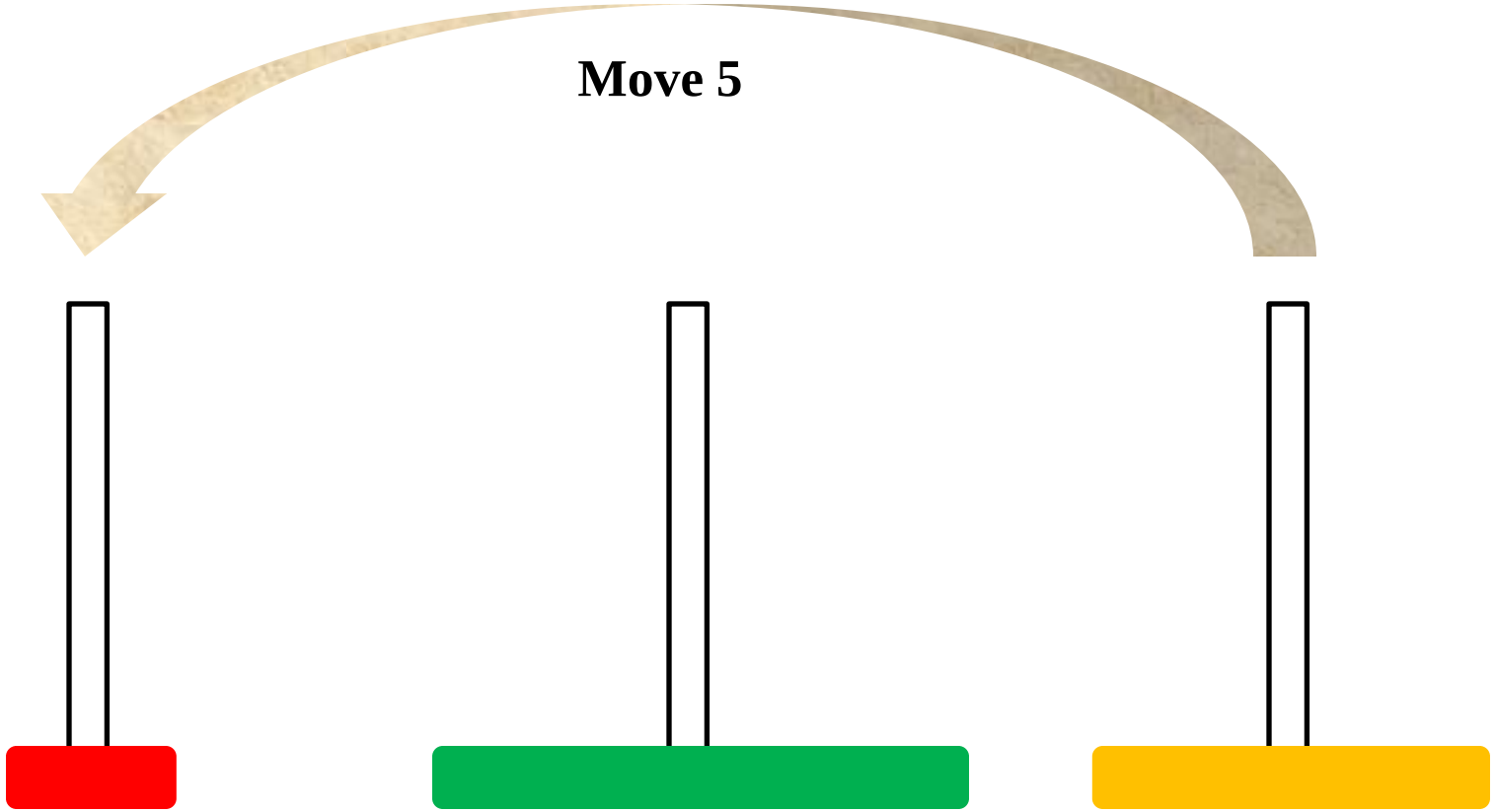

Peg 1
Peg 2
Peg 3

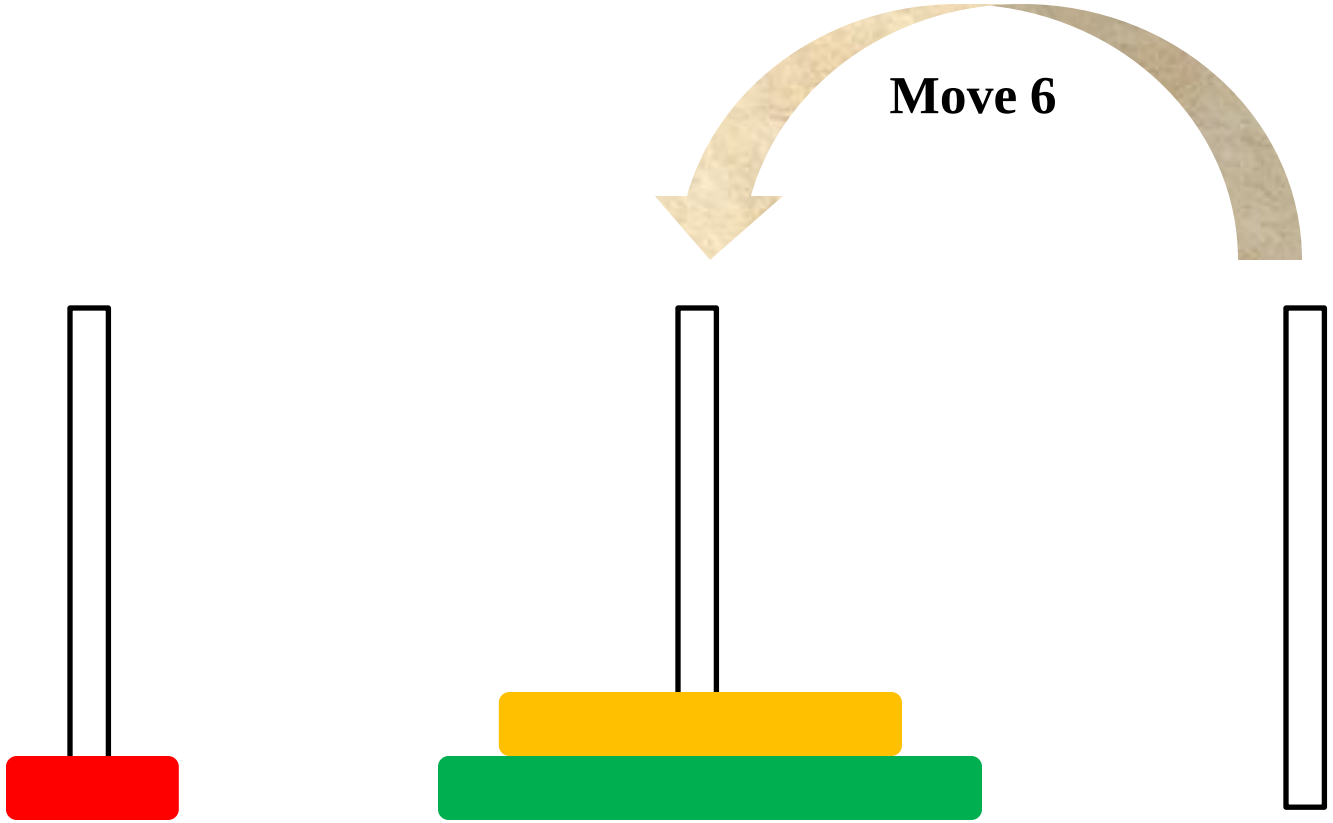

Move 1

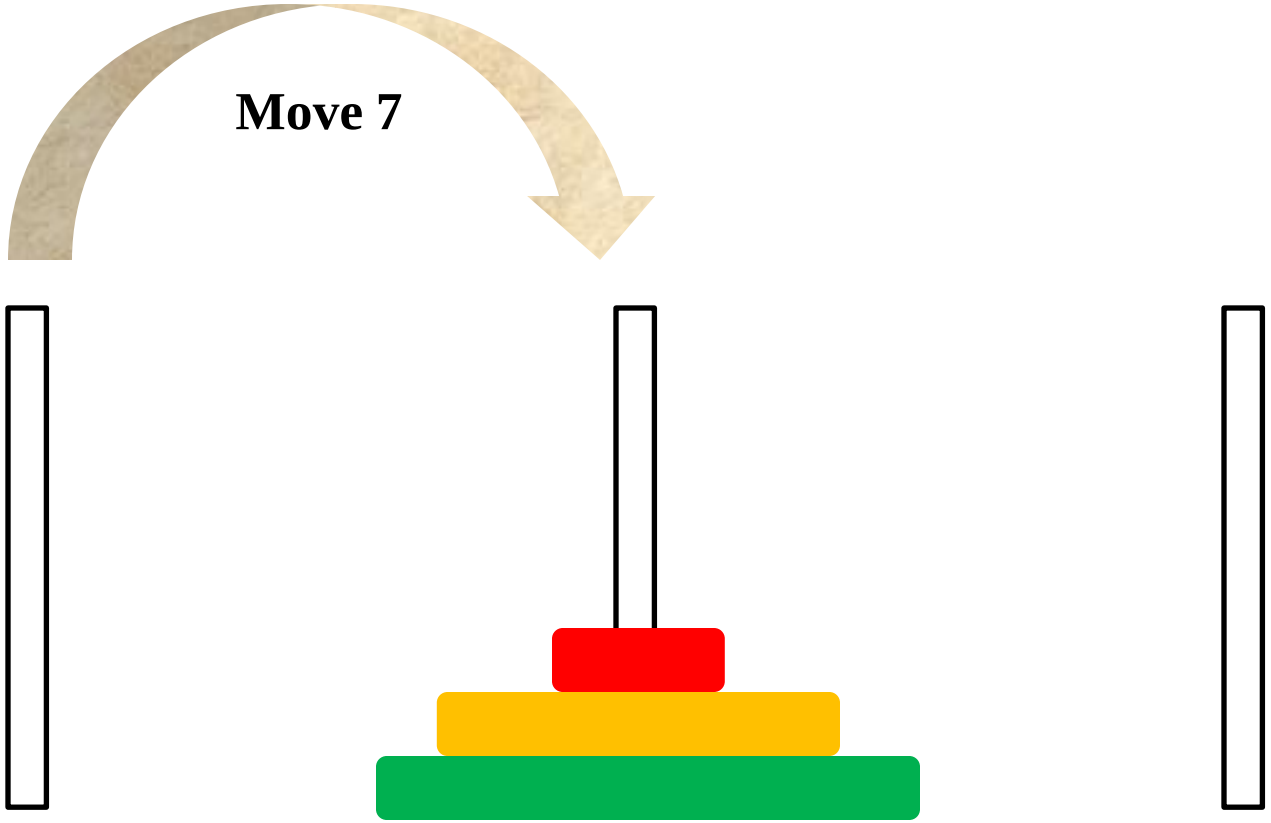

Move 2


Move 3

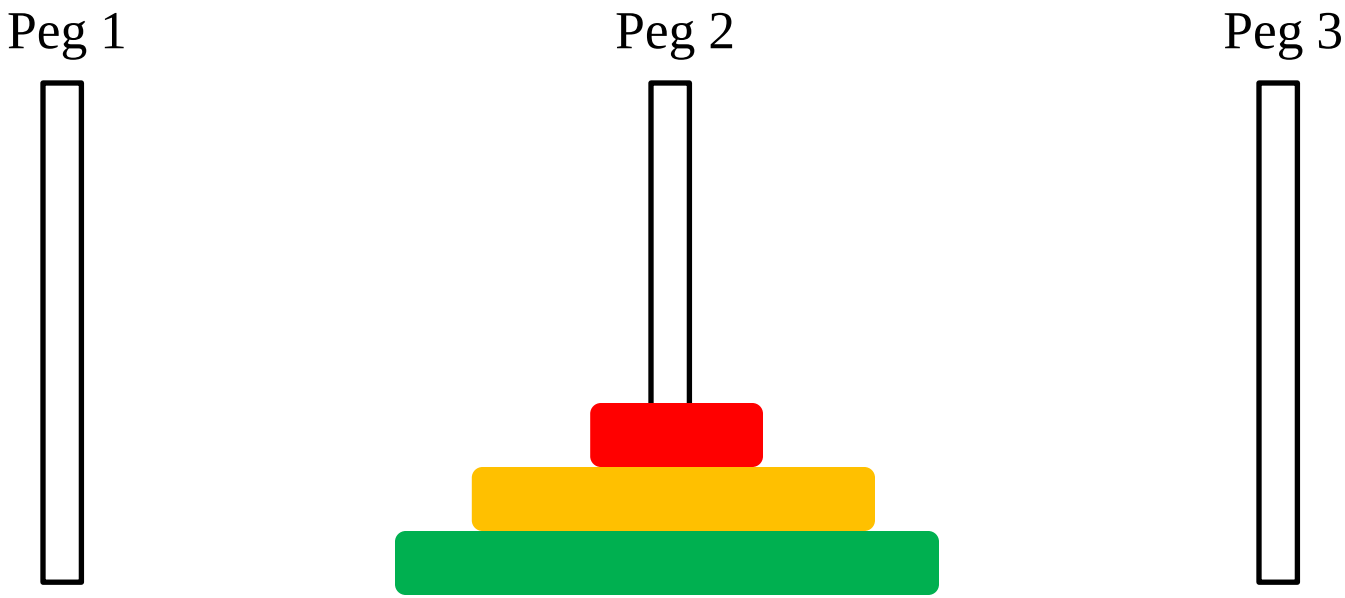

Move 4


Move 5


Move 6


Move 7

Steps Completed!



Done moving n = 3 disks from peg 1 to peg 2.
Total no. of moves required = 7

⇒ Let n denote the total no. of disks resting on peg 1.

⇒ Let C_n denote the total no. of moves required to transfer n disks from peg 1 to peg 2 is:

$$C_n = 2^n - 1,\ n>1$$

For 1 disk,

$$C_1 = 1$$

For 2 disk,

$$C_2 = 3$$

For 3 disk,

$$C_3 = 7 \text{ and so on.}$$

⇒ The recurrence relation for the sequence $C_1, C_2, \ldots, C_n$, where C_n denotes no. of moves in tower of Hanoi puzzle is given as:

$$C_n = 2C_{n-1} + 1\ ,\ n>1$$

with initial condition

$$C_1 = 1$$

Solving Recurrence Relations

→ To solve a recurrence relation having the sequence

$$a_0, a_1, a_2, \ldots\ldots, a_n$$

→ Is to find an explicit formula for the general term a_n.

→ In this section, we discuss two methods that applies to linear homogenous recurrence relation with constant coefficients.

→ The methods are:-

- ❑ Solution of Recurrence relations by iterations
- ❑ Special method that applies to linear homogeneous recurrence relations with constant coefficients.

Solving Recurrence Relations by Iterations

Example 1:

Solve $a_n = a_{n-1} + 3$ by iteration with initial condition $a_1 = 2$.

Solution:

We have,

$$a_n = a_{n-1} + 3 \quad \text{.....(i)}$$

Replacing n by n-1, we get,

$$a_{n-1} = a_{n-2} + 3$$

Substituting the value of a_{n-1} in equation (i), we get,

$$a_n = a_{n-2} + 2.3 \quad \text{.....(ii)}$$

Replacing n by n-2, in equation (i), we get,

$$a_{n-2} = a_{n-3} + 3$$

Substituting the value of a_{n-2} in equation (ii), we get,

$$a_n = a_{n-2} + 3.3$$

In general,

$$a_n = a_{n-k} + k.3$$

$$a_n = a_{n\text{-k}} + k.3$$

If $k = n$-1,

$$a_n = a_1 + (n\text{-}1).3$$

Which is the required solution for the given Recurrence Relation.

Example 2:

Solve the recurrence relation, $S_n = 2S_{n-1}$ by iteration with initial condition $S_0 = 1$.

Solution:

We have,

$$S_n = 2S_{n-1} \quad \ldots\ldots\ldots\text{(i)}$$

Replacing n by n-1, we get,

$$S_{n-1} = 2S_{n-2}$$

Substituting the value of S_{n-1} in equation (i), we get,

$$S_n = 2^2 S_{n-2} \quad \ldots\ldots\ldots\text{(ii)}$$

Replacing n by n-2, in equation (i), we get,

$$S_{n-2} = 2S_{n-3}$$

Substituting the value of S_{n-2} in equation (ii), we get,

$$S_n = 2^3 S_{n-3}$$

In general,

$$S_n = 2^k S_{n-\text{k}}$$

$$S_n = 2^k S_{n\text{-k}}$$

If $k = n$,

$$S_n = 2^n S_0$$

Which is the required solution for the given Recurrence Relation.

Example 3: *Tower of Hanoi Puzzle Solution*

Solve the recurrence relation, $C_n = 2C_{n-1} + 1$ by iteration with the initial condition $C_1 = 1$.

Solution:

We have,

$$C_n = 2C_{n-1} + 1 \text{ …….(i)}$$

Replacing n by n-1, we get,

$$C_{n-1} = 2C_{n-2} + 1$$

Substituting the value of C_{n-1} in equation (i), we get,

$$C_n = 2(2C_{n-2} + 1) + 1$$

$$C_n = 2^2 C_{n-2} + 2 + 1 \text{ …….(ii)}$$

Replacing n by n-2, in equation (i), we get,

$$C_{n-2} = 2C_{n-3} + 1$$

Substituting the value of C_{n-2} in equation (ii), we get,

$$C_n = 2^3 C_{n-3} + 3 + 2 + 1$$

In general,

$$C_n = 2^k C_{n-k} + 2^{k-1} + 2^{k-2} + \ldots\ldots\ldots\ldots\ldots + 2^1 + 1$$

If $k = n - 1$,

$$C_n = 2^{n-1} C_1 + 2^{n-2} + 2^{n-3} + \ldots\ldots\ldots\ldots\ldots + 2^1 + 1$$

or, $C_n = 2^{n-1} + 2^{n-2} + 2^{n-3} + \ldots\ldots\ldots\ldots\ldots + 2^1 + 1$

or, $C_n = \dfrac{1(2^n - 1)}{(2-1)}$

$$\therefore C_n = 2^n - 1$$

Which is the required solution for the given Recurrence Relation.

Linear Homogeneous Recurrence Relation with constant coefficients

→ A linear homogeneous recurrence relation of order k with constant coefficients is a recurrence relation of the form:

$$a_n = c_1 a_{n-1} + c_2 a_{n-2} + c_3 a_{n-3} + \ldots \ldots + c_k a_{n-k},\ c_k \neq 0$$

→ Notice that a linear homogeneous recurrence relation of order k with constant coefficients with the k initial conditions

$$a_0 = c_0,\ a_1 = c_1,\ \ldots\ldots, a_{k-1} = c_{k-1}$$

uniquely defines a sequence.

Exercise

Determine whether the given recurrence relations are linear homogeneous recurrence relation with const. coeffs. or not.

Exercise 1: $S_n = 2S_{n-1}$

Answer: The given recurrence relation is a linear homogeneous recurrence relation of order 1.

Exercise 2: $f_n = f_{n-1} + f_{n-2}$

Answer: The given recurrence relation is a linear homogeneous recurrence relation of order 2.

Exercise 3: $a_n = 3a_{n-1} . a_{n-2}$

Answer: The given recurrence relation is a not a linear homogeneous recurrence with constant coefficient.

Exercise 4: $a_n - a_{n-1} = 2n$

Answer: The given recurrence relation is a not a linear homogeneous recurrence with constant coefficient because the expression on the right hand side is not zero.

Exercise 5: $a_n = 3n\, a_{n-1}$

Answer: The given recurrence relation is a not a linear homogeneous recurrence relation with constant coefficient because coeff $3n$ is not const.

Theorem

Let

$$a_n = c_1 a_{n-1} + c_2 a_{n-2} \quad \ldots\ldots\ldots\ldots\ldots\ldots\text{(i)}$$

be a second order linear homogeneous recurrence relation with constant coefficients.

If S and T are solutions of (i), then,

$$U = bS + dT$$

is also a solution of equation (i).

If r is the root of equation

$$t^2 - c_1 t - c_2 = 0 \quad \ldots\ldots\ldots\ldots\ldots\ldots\text{(ii)}$$

Then the sequence $\{r_n\}$, $n = 0,1,\ldots$ is a solution of (i).

If a_n is a sequence defined by (i), then

$$a_0 = c_0,\ a_1 = c_1 \quad \ldots\ldots\ldots\ldots\ldots\ldots\text{(iii)}$$

And r_1 and r_2 be the root of (ii) with $r_1 \neq r_2$, then there exists a constant b and d such that

$$a_n = br_1^n + dr_2^n, n = 0,1,2,\ldots..$$

Examples

Example 1: Assume that the deer population of Rustic country is 200 at the time $n = 0$ and 220 at time $n = 1$ and that the increase from time n-1 to time n is twice the increase from time n-2 to n-1. Write the recurrence relation and an initial condition that define the deer population at time n and then solve the recurrence relation.

Solution:

Let d_n denote the deer population at time n.

We have the initial conditions,

d_0=200, d_1 = 220

By question,

$$d_n - d_{n-1} = 2\,(d_{n-1} - d_{n-2})$$

$$d_n = 3\,d_{n-1} - 2d_{n-2} \quad \text{.....(i)}$$

Let, S and T be the solutions of the Recurrence Relation (i), then

$$t^n = 3t^{n-1} - 2t^{n-2}$$

Dividing both sides by t^{n-2}, we get,

$$t^2 - 3t + 2 = 0$$

$or,\ t^2 - 2t - t + 2 = 0$

$or,\ t\,(t-2) - 1(t-2) = 0$

$or,\ (t-1).(t-2) = 0$

$\therefore t = 1, 2$ [i.e. $r_1 = 1$, $r_2 = 2$ and $r_1 \neq r_2$]

Then, $S = r_1^n = 1^n$ and $T = r_2^n = 2^n$

The solution is the sequence d_n of the form:

$$d_n = b.1^n + c.2^n$$

Using initial conditions,

$$d_0 = b.1^0 + c.2^0$$

$$\therefore b + c = 200$$

$$d_1 = b.1^1 + c.2^1$$

$$\therefore\ b + 2c = 220$$

Solving, we get, $b = 180$, $c = 20$

$$\therefore\ d_n = 180 + 20 \times 2^n$$

Example 2: Find an explicit formula for the Fibonacci sequence.

Solution:

The Fibonacci sequence is defined by the second order linear homogeneous recurrence relation,

$$f_n = f_{n-1} + f_{n-2}\ ,\ n \geq 3 \qquad \text{.....(i)}$$

With initial conditions, $f_1 = 1$ and $f_2 = 1$

$$\text{or, } f_n - f_{n-1} - f_{n-2} = 0$$

To solve the above recurrence relation (i), we have to first solve the quadratic equation,

$$t^2 - t - 1 = 0$$

Solving for t, we get,

$$t = \frac{-(-1) \pm \sqrt{(-1)^2 - 4 \times 1 \times -1}}{2 \times 1}$$

$$\therefore t = \frac{1 \pm \sqrt{5}}{2}$$

$$\therefore r_1 = \frac{1+\sqrt{5}}{2}, \text{and } r_2 = \frac{1-\sqrt{5}}{2}$$

Then,

$$S = r_1^n = \left(\frac{1+\sqrt{5}}{2}\right)^n \text{ and } T = r_2^n = \left(\frac{1-\sqrt{5}}{2}\right)^n$$

Thus, the solution is of the form,

$$f_n = bS + dT$$

$$f_n = b\left(\frac{1+\sqrt{5}}{2}\right)^n + d\left(\frac{1-\sqrt{5}}{2}\right)^n$$

Using initial conditions for $n = 1$ and $n = 2$

$$1 = b\left(\frac{1+\sqrt{5}}{2}\right)^1 + d\left(\frac{1-\sqrt{5}}{2}\right)^1 \qquad \text{.....(ii)}$$

$$1 = b\left(\frac{1+\sqrt{5}}{2}\right)^2 + d\left(\frac{1-\sqrt{5}}{2}\right)^2 \qquad \text{.....(iii)}$$

Rewriting equations (ii) and (iii), we get,

$$1 = \frac{b}{2} + \frac{b\sqrt{5}}{2} + \frac{d}{2} - \frac{d\sqrt{5}}{2} \qquad \text{.....(iv)}$$

$$1 = \frac{b}{4} + \frac{b\sqrt{5}}{2} + \frac{5b}{4} + \frac{d}{4} - \frac{d\sqrt{5}}{2} + \frac{5d}{4} \qquad \text{.....(v)}$$

Subtracting equation (iv) from (v), we get,

$$0 = -\frac{b}{4} + 0 + \frac{5b}{4} - \frac{d}{4} + \frac{5d}{4}$$

$$0 = b + d$$

$$\therefore b = -d \qquad \text{.....(vi)}$$

From equations (vi) and (ii), we get,

$$1 = -d\left(\frac{1+\sqrt{5}}{2}\right) + d\left(\frac{1-\sqrt{5}}{2}\right)$$

$$or, 1 = -\frac{d}{2} - \frac{d\sqrt{5}}{2} + \frac{d}{2} - \frac{d\sqrt{5}}{2}$$

$$or, 1 = -d\sqrt{5}$$

$$\therefore d = -\frac{1}{\sqrt{5}}$$

$$\therefore b = -d = \frac{1}{\sqrt{5}}$$

$\therefore$ The explicit formula for the Fibonacci sequence is:

$$f_n = \frac{1}{\sqrt{5}}\left(\frac{1+\sqrt{5}}{2}\right)^n - \frac{1}{\sqrt{5}}\left(\frac{1-\sqrt{5}}{2}\right)^n$$

Thank You!

Any Queries?

Bachelor of Engineering (B.E.) in
Computer Engineering (BCT)
Electronics, Communication & Information Engineering (BEI)
Third Year Second Part
Elective II Course

Analysis of Algorithms

Chapter 1. Introduction to Algorithm Analysis

Session 3

by:

Er. Bikal Adhikari

(*B.E., M.Sc. Engg., M.Sc. ISE*)

Assistant Professor, Dept. of Electronics & Computer Engg.

Pulchowk Campus, IOE, TU

CHAPTER OUTLINE

Chapter 1. Introduction to Algorithm Analysis

5 hours

1.1 Algorithm and its properties, RAM model, time and space complexity, detailed analysis of algorithms, Concept of Aggregate Analysis

1.2 Asymptotic notations (Big-O, Big-Ω and Big-Θ), their geometrical interpretations and examples

1.3 Concept of best case, average case and worst case performance of an algorithm

1.4 Modeling algorithms by recurrence relation

1.5 Solving recurrence relation for evaluating computational complexity

- 1.5.1 Recursion tree method
- 1.5.2 Substitution method
- 1.5.3 Using masters theorem

RECURRENCES

→ A recurrence is a function that is defined in terms of

- One or more base cases, and
- Itself, with smaller arguments

→ Examples:

$$T(n) = \begin{cases} 1 & \text{if } n = 1, \\ T(n-1) + 1 & \text{if } n > 1. \end{cases}$$

Solution:

$T(n) = n.$

$$T(n) = \begin{cases} 1 & \text{if } n = 1, \\ 2T(n/2) + n & \text{if } n \geq 1. \end{cases}$$

Solution:

$T(n) = n \lg n + n.$

$$T(n) = \begin{cases} 0 & \text{if } n = 2, \\ T(\sqrt{n}) + 1 & \text{if } n > 2. \end{cases}$$

Solution:

$$T(n) = \lg\lg n.$$

$$T(n) = \begin{cases} 1 & \text{if } n = 1, \\ T(n/3) + T(2n/3) + n & \text{if } n > 1. \end{cases}$$

Solution:

$$T(n) = \Theta(n \lg n).$$

METHODS OF SOLVING RECURRENCES

→ Following are the different methods of solving recurrences:

i. Recursion Trees
ii. Iteration Method
iii. Substitution Method
iv. Master Method
v. Characteristic equation method ----Done in Discrete Structure

Modeling Algorithms by Recurrence Relation

When we model algorithms by recurrence relations, we're essentially translating the *divide-and-conquer* or *recursive structure* of an algorithm into a mathematical equation that describes its running time. This is one of the most powerful tools in Analysis of Algorithms because it allows us to reason about complexity systematically.

In the following slides, we will explore the concept.

How to Form a Recurrence Relation?

Example 1:

→ Let us consider following function:

```
void Test(int n)                    ----------  T(n)
{
    if(n>0)
    {
    printf("%d",n);                 ----------  1
    Test(n-1);                      ----------  T(n-1)
    }
}
```

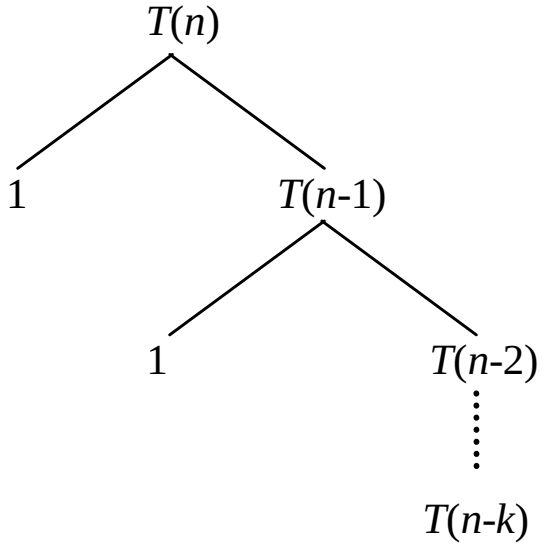
→ The recurrence relation and the recurrence tree are given below:

$T(n)=T(n-1)+1$

With initial condition:

$T(0)=1$

$$T(n)=\begin{cases}1 & ,n=0\\ T(n-1)+1 & ,n>0\end{cases}$$



→ The solution to the recurrence relation is:

$$T(n) = 1 + 1 + \ldots\ldots + k \text{ times} + T(n-k) = k + T(n-k)$$

For $n = k$

$$\therefore T(n) = n + T(0) = n + 1 = O(n)$$

Example 2:

→ Let us consider following function:

```
void Test(int n)
{
    if(n>0)
    {
        for(i=0;i<n;i++)
        {
            printf("%d",n);
        }
    Test(n-1);
    }
}
```

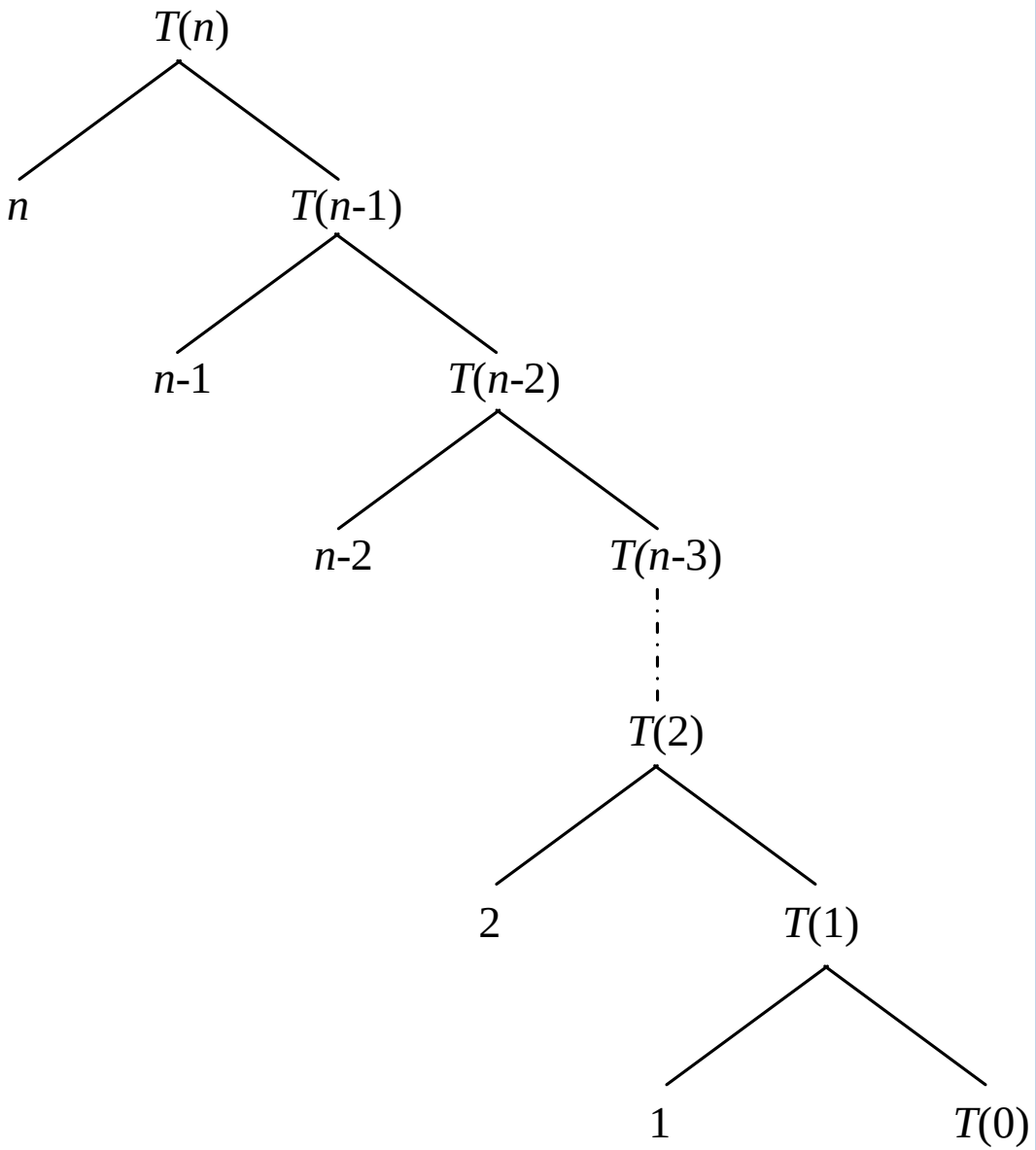
→ The recurrence relation for the given algorithm is:

$$T(n) = T(n-1) + n$$

→ With initial condition, the recurrence relation can be written as:

$$T(n) = \begin{cases} 1 & ,n = 0 \\ T(n-1) + n & ,n > 0 \end{cases}$$

→ The recursion tree for the given recurrence relation is:



The solution is:

$$T(n) = 1 + 1 + 2 + \ldots\ldots + n - 1 + n$$

$$\therefore T(n) = \frac{n(n+1)}{2} = O(n^2)$$

Example 3:

→ Let us consider following function:

```
void Test(int n)
{
    if(n>0)
    {
        for(i=0;i<n;i=i*2)
        {
            printf("%d",i);
        }
    Test(n-1);
    }
}
```

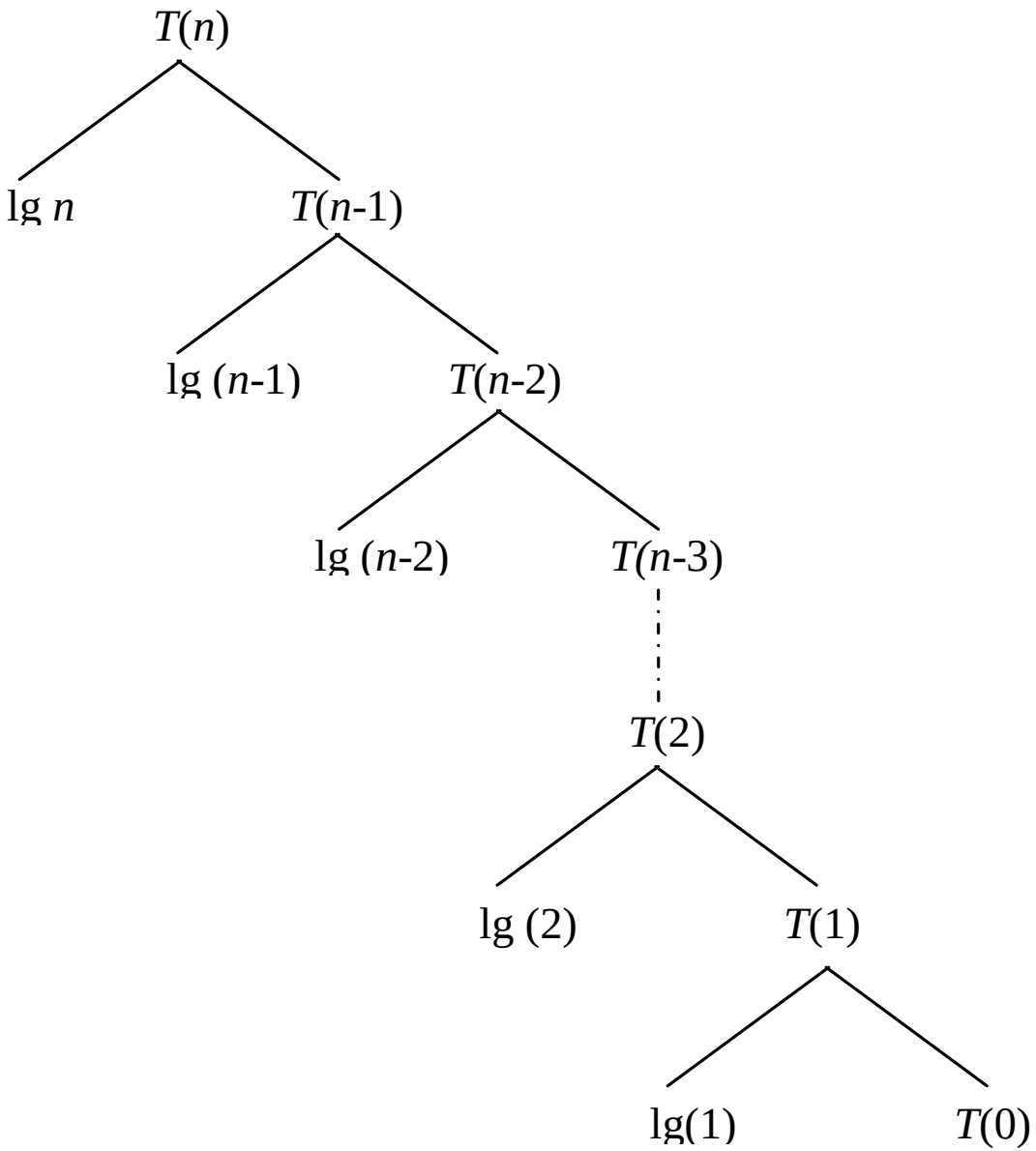
→ The recurrence relation for the given algorithm is:

$$T(n) = T(n\text{-}1) + \lg n$$

→ With initial condition, the recurrence relation can be written as:

$$T(n) = \begin{cases} 1 & ,n = 0 \\ T(n-1) + \lg n & ,n > 0 \end{cases}$$

→ The recursion tree for the given recurrence relation is:



→ The solution is:

$$T(n) = 1 + \lg 1 + \lg 2 + \ldots\ldots + \lg(n-1) + \lg n$$
$$\therefore T(n) = 1 + \lg\left(1 \times 2 \times 3 \times \ldots\ldots \times (n-1) \times n\right)$$
$$= 1 + \lg(n!) = O(n \lg n)$$

Example 4:

→ Let us consider following function:

```
void Test(int n)
{
    if(n>1)
    {
        for(i=0;i<n;i++)
        {
            statement;
        }
    Test(n/2);
    Test(n/2);
    }
}
```

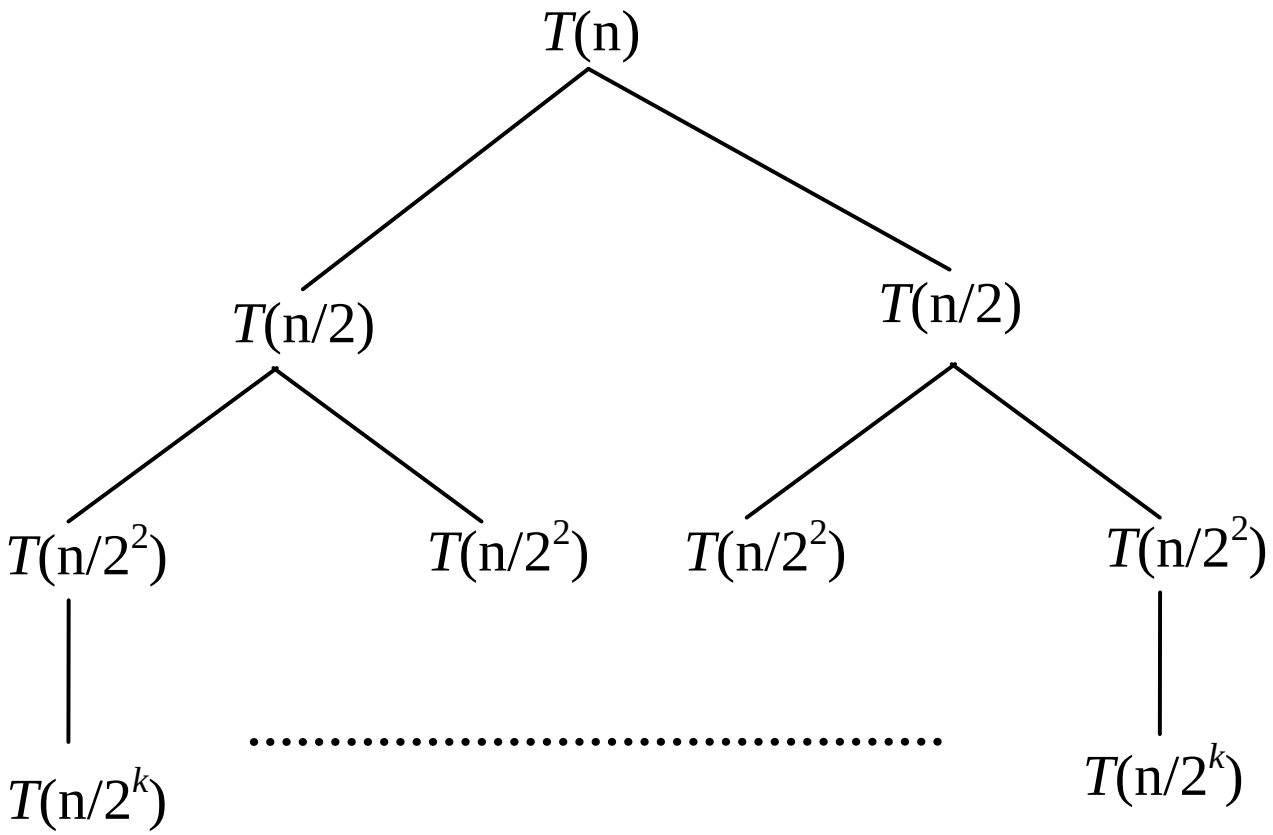
→ The recurrence relation for the given algorithm is:

$$T(n) = 2T(n/2) + n$$

→ With initial condition, the recurrence relation can be written as:

$$T(n) = \begin{cases} 1 & ,n = 1 \\ 2T(n/2) + n & ,n > 1 \end{cases}$$

→ The recursion tree for the given recurrence relation is:



→ In general:

$$\frac{n}{2^k} = 1$$

$$or, n = 2^k$$

$$\therefore k = \lg n$$

→ Total cost from recursion tree, $T(n) = k \times n = n \lg n$

→ Therefore, $T(n) = O(n \lg n)$

ITERATION METHOD

→ Iteration method uses following steps:

i. Expand the relation such that a summation independent of n is obtained.

ii. Bound the summation.

→ **Example 1**: Solve the following recurrence relation usign iteration method

$$T(n) = \begin{cases} 1 & ,n = 0 \\ T(n-1) + n & ,n > 0 \end{cases}$$

Solution:

$T(0) = 1$

$T(n) = T(n-1) + n$

Similarly,

$T(n-1) = T(n-2) + (n-1)$

Substituting the expression of $T(n\text{-}1)$ in that of $T(n)$, we get,

$T(n) = T(n\text{-}2) + n\text{-}1 + n$

Again, we may write,

$T(n-2) = T(n-3) + (n-2)$

Substituting it in the expression of T(n), we get,

$$T(n) = T(n\text{-}3) + (n-2) + (n\text{-}1) + n$$

In general,

$$T(n) = T(n\text{-}k) + (n-k+1) + \ldots + (n\text{-}1) + n$$

For $n = k$,

$$T(n) = T(0) + 1 + 2 + \ldots\ldots + (n-1) + n = 1 + \frac{n.(n+1)}{2}$$

$$\therefore T(n) = O(n^2)$$

Example 2: Solve the following recurrence relation using iteration method

$$T(n) = \begin{cases} 1 & ,n = 0 \\ T(n-1)+1 & ,n > 0 \end{cases}$$

Solution:

$T(0) = 1$

$T(n) = T(n-1) + 1$

Similarly,

$T(n-1) = T(n-2) + 1 + 1$

Substituting the expression of $T(n\text{-}1)$ in that of $T(n)$, we get,

$T(n) = T(n\text{-}2) + 2$

Again, we may write,

$T(n-2) = T(n-3) + 1$

Substituting the value of $T(n\text{-}2)$ in the expression of $T(n)$, we get

$T(n) = T(n\text{-}3) + 3$

In general,

$T(n) = T(n\text{-}k) + k$

Assuming $n = k$, we get,

$$T(n) = T(0) + n = 1 + n$$

$$\therefore T(n) = O(n)$$

SUBSTITUTION METHOD

→ Substitution method uses following steps:

i. Guess the solution

ii. Use induction to find the constants and show that the solution works.

→ Example:

$$T(n) = \begin{cases} 1 & \text{if } n = 1, \\ 2T(n/2) + n & \text{if } n \geq 1. \end{cases}$$

Guess :

$T(n) = n \lg n + n.$

Induction

Basis : $n = 1 \Rightarrow n \lg n + n = 1 = T(n)$

Inductive Step :

Inductive hypothesis is that $T(k) = k \lg k + k$ for all $k < n$

We'll use this inductive hypothesis for $T(n/2)$

$$
\begin{aligned}
T(n) &= 2T(n/2) + n \\
&= 2\left(\frac{n}{2}\lg\frac{n}{2} + \frac{n}{2}\right) + n \quad \text{(by inductive hypothesis)} \\
&= n\lg\frac{n}{2} + n + n \\
&= n(\lg n - \lg 2) + n + n \\
&= n\lg n - n + n + n \\
&= n\lg n + n.
\end{aligned}
$$

MASTER METHOD

Master Theorem:

Let $a \geq 1$ and $b > 1$ be constants, let $f(n)$ be a function,

and let $T(n)$ be defined on the non - negative integers by the recurrence

$T(n) = aT(n/b) + f(n)$,

where we interpret n/b to mean either $\lfloor n/b \rfloor$ or $\lceil n/b \rceil$

Then $T(n)$ has the following asymptotic bounds :

Case 1 : If $f(n) = O(n^{\log_b a - \epsilon})$ for some constant $\epsilon > 0$, then

$T(n) = \Theta\left(n^{\log_b a}\right)$

Case 2 : If $f(n) = \Theta\left(n^{\log_b a}\right)$, then

$T(n) = \Theta\left(n^{\log_b a} \lg n\right)$

Case 3 : If $f(n) = \Omega\left(n^{\log_b a + \epsilon}\right)$ for some constant $\epsilon > 0$,

and if $af(n/b) \leq cf(n)$ for some constant $c < 1$ and all sufficiently

large n, then

$T(n) = \Theta\left(f(n)\right)$

MASTER METHOD

***T*(*n*):** The total running time of an algorithm
***a*:** The no. of sub-problems the algorithm divides the problem into.
***n*/*b*:** The size of each sub problem. The problem is split into smaller pieces, each of size n/b.
***f*(*n*):** The cost of work done outside the recursive calls- for example splitting the problem, merging results, or performing additional computations

Case 1: Too many leaves

If the no. of leaf nodes outweighs the sum of the internal evaluation cost, the total running time is $o(n^{\log_b a})$

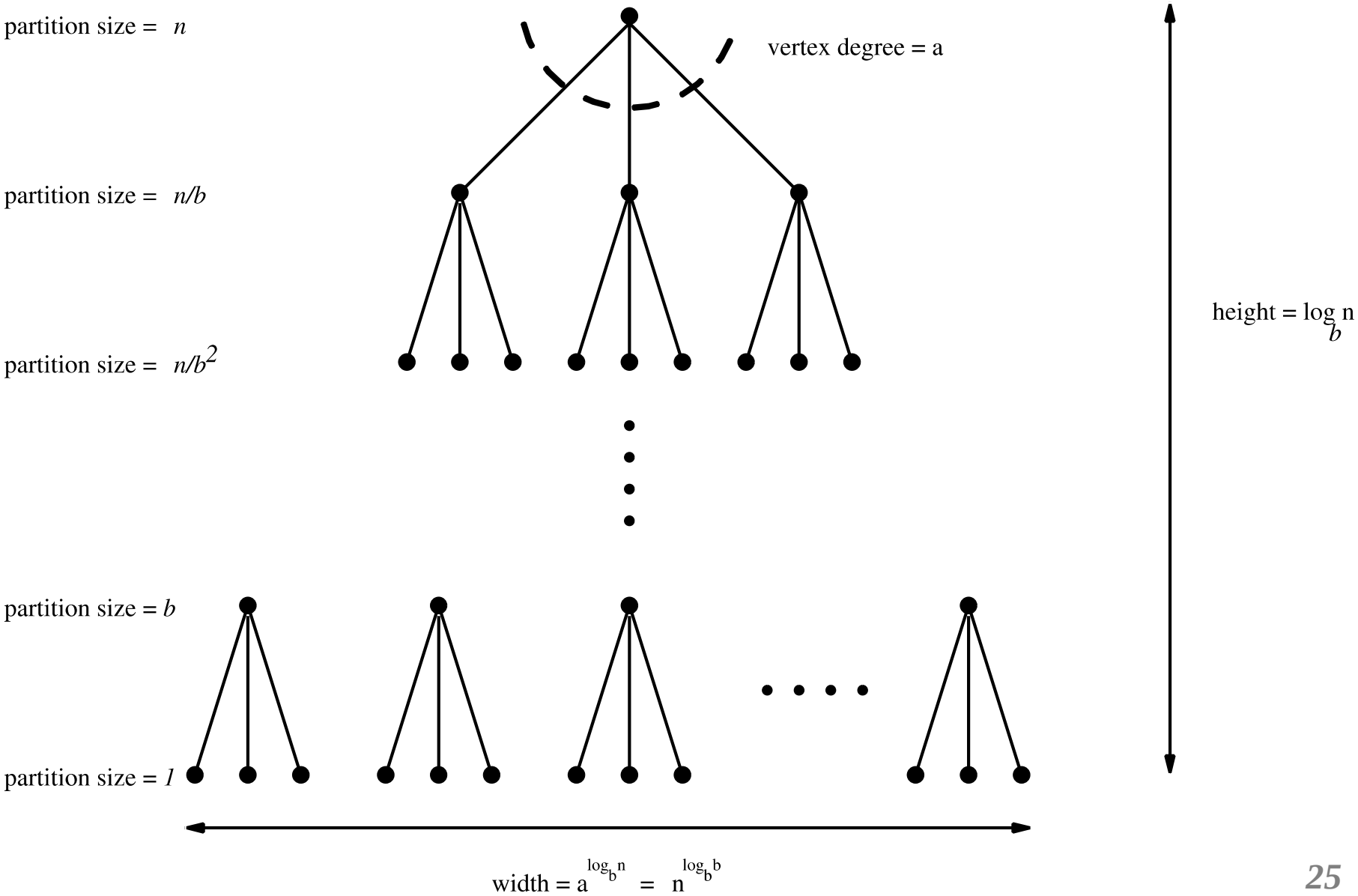
Case 2: Equal work per level

As we move down the tree, each problem gets smaller but there are more of them to solve. If the sum of the internal evaluation costs at each level are equal the total running time is the cost per level $o(n^{\log_b a}.\lg n)$

Case 3: Too expensive a root

If the internal evaluation costs grow rapidly enough with n, then the cost of the root evaluation may dominate. If so, the total running time is $O(f(n))$

MASTER METHOD: RECURSION TREE VISUALIZATION



EXAMPLES OF MASTER METHOD:

Q_1. Find the solution of following recurrence relation using masters theorem:

$$T(n) = 7T(n/2) + cn^2$$

Solution:

Given recurrence relation is:

$$T(n) = 7T(n/2) + cn^2$$

Comparing it with standard form:

$$T(n) = aT(n/b) + f(n)$$

We get,

$$\text{a} = 7, \text{b} = 2,\ f(n) = cn^2$$

$$n^{\log_b a} = n^{\log_2 7} = n^{2.8}$$

$$cn^2 = O(n^{\log_b a - \epsilon}), \text{for any} \in = 0.8 (\in > 0)$$

$$\therefore T(n) = \Theta\left(n^{\log_b a}\right) = \Theta\left(n^{2.8}\right)$$

Q_2. Find the solution of following recurrence relation using masters theorem:

$$T(n) = 2T(n/2) + cn$$

Solution:

Given recurrence relation is:

$$T(n) = 2T(n/2) + cn$$

Comparing it with standard form:

$$T(n) = aT(n/b) + f(n)$$

We get,

$a = 2, b = 2, f(n) = cn$

$\Rightarrow n^{\log_b a} = n^{\log_2 2} = n$

$f(n) = cn = \Theta(n^{\log_b a})$

$\therefore T(n) = \Theta\left(n^{\log_b a} \lg n\right) = \Theta\left(n \lg n\right)$

Q_3. Find the solution of following recurrence relation using masters theorem:

$$T(n) = 4T(n/2) + n^3$$

Solution:

Given recurrence relation is:

$$T(n) = 4T(n/2) + n^3$$

Comparing it with standard form: $T(n) = aT(n/b) + f(n)$

We get,

$a = 4, b = 2, f(n) = n^3$

$n^3 = \Omega\left(n^{\log_b a + \epsilon}\right) = \Omega\left(n^{\log_2 4 + \epsilon}\right)$ for any $\epsilon > 0$

Also,

$a.f(n/b) \leq c.f(n)$

$or, 4.(n/2)^3 \leq cn^3$

$or, \frac{1}{2}n^3 \leq cn^3$ True for $c \geq \frac{1}{2}$

$\therefore T(n) = \Theta(f(n)) = \Theta\left(n^3\right)$

Q_4. Find the solution of following recurrence relation using masters theorem:

$$T(n) = 2T(n/2) + n \lg n$$

Solution: Given recurrence relation is: $T(n) = 2T(n/2) + n \lg n$

Comparing it with standard form: $T(n) = aT(n/b) + f(n)$

We get,

$$a = 2, b = 2, f(n) = n \lg n$$

$$\Rightarrow n^{\log_b a} = n^{\log_2 2} = n$$

even though it appears to have the proper form: a = 2, b = 2, f(n) = n lg n, and $o(n^{\log_b a})$= n. You might mistakenly think that case 3 should apply, since f(n) = n lg n is asymptotically larger than $o(n^{\log_b a})$ = n. The problem is that it is not polynomially larger. The ratio f(n)/ $n^{\log_b a}$ = (n lg n)/n = lg n is asymptotically less than n^ε for any positive constant ε. Consequently, the recurrence falls into the gap between case 2 and case 3. So we cannot use master method to solve this R-R. We can use other method to solve this R-R.

Thank You!!!